

SUPSI – Scuola Universitaria Professionale della Svizzera Italiana
Dipartimento Tecnologie Innovative (DTI)

Progetto di Semestre

Relatore: Prof. Loris Cannelli

In collaborazione con Politecnico di Milano

Battery GEIS MLOps Platform

*Una pipeline MLOps end-to-end per la spettroscopia di impedenza
elettrochimica galvanostatica su celle Litio-Ione*

Davide Corso & Marco Soldani

Anno Accademico 2025–2026

28 aprile 2026

Abstract

Il presente progetto di semestre, svolto in collaborazione con il Politecnico di Milano, ha per oggetto la **predizione di parametri di celle Litio-Ione** caratterizzate tramite spettroscopia di impedenza elettrochimica galvanostatica (GEIS) su una pouch LiCoO_2 da 10 A h. Il dataset, fornito dal gruppo di ricerca del Politecnico, conta 9805 misurazioni organizzate come **5 livelli di invecchiamento $\times$ 8 temperature $\times$ 5 SOC** e ~ 49 frequenze logaritmicamente spaziate, ovvero **200 curve di Nyquist complete**.

Il **compito primario** consisteva nello sviluppare tre notebook Jupyter contenenti modelli di machine learning per altrettanti task predittivi:

1. **Task 1 — Leave-One-Out (LOO):** ricostruzione di una singola coppia (Aging, Temperatura) a partire dalle altre 39 (regressione multi-output).
2. **Task 2 — Young/Old Classification:** dati un livello di Aging e un livello di SOC, predire la classe *young* ($\text{Aging} \leq 2$) vs *old* ($\text{Aging} \geq 3$) delle **8 curve di Nyquist** (una per temperatura) corrispondenti a quella coppia, tenute fuori dall’addestramento.
3. **Task 3 — Aging Interpolation:** predizione di un *intero* livello di invecchiamento mai osservato in addestramento.

I migliori modelli ottenuti su un benchmark di cinque/sei stimatori sono rispettivamente *KNN distance-weighted* ($R^2 = 0.991$, Task 1), *SVM con kernel RBF* (accuracy = 1.000 sulle 8 curve del hold-out di default ($\text{Aging} = 4, \text{SOC} = 3$), Task 2) e di nuovo *KNN distance-weighted* ($R^2 = 0.986$, Task 3). I risultati sono prodotti da una pipeline interamente riproducibile (random seed fissato, splitter coerenti con il task) e tracciata con MLflow.

Estensione volontaria. Sebbene la consegna fosse limitata ai notebook, abbiamo costruito sopra di essi una piccola piattaforma MLOps end-to-end – non richiesta – per esporre i risultati a un utente non-developer e per esplorare nella pratica le prassi di settore: configurazione centralizzata in YAML, logging strutturato con rotazione, suite di 81 test pytest con coverage gate al 90 % in CI (attuale: 95 %), scansioni di sicurezza (**bandit**, **pip-audit**), build Docker multistage, backend **FastAPI** con frontend React per il *serving* e un modulo di **drift detection** basato su test di KS e PSI.

Il report descrive in dettaglio il dominio fisico, il dataset, la metodologia condivisa fra i tre task, i risultati ottenuti e – in appendice concettuale – l’architettura della piattaforma volontaria, chiudendosi con un’analisi dei limiti residui e delle direzioni di sviluppo futuro.

Abstract (English)

This semester project, carried out in collaboration with Politecnico di Milano, focuses on the **prediction of Lithium-Ion battery parameters** characterized via galvanostatic electrochemical impedance spectroscopy (GEIS) on a 10 A h LiCoO₂ pouch cell. The dataset, supplied by the Politecnico research group, contains 9805 measurements organized as **5 aging levels** × **8 temperatures** × **5 SOC** and ~49 logarithmically spaced frequencies, i.e. **200 complete Nyquist curves**.

The **primary deliverable** consisted of three Jupyter notebooks implementing machine-learning models for three predictive tasks: **(1)** Leave-One-Out reconstruction of a single (*Aging*, *Temperature*) pair from the other 39 (multi-output regression); **(2)** given an Aging level and a SOC level, predict the *young* (*Aging* ≤ 2) vs *old* (*Aging* ≥ 3) class of the **8 Nyquist curves** (one per temperature) corresponding to that pair, held out from training; **(3)** interpolation of an entire unseen aging level (multi-output regression).

The best models on a benchmark of five/six estimators are respectively *KNN distance-weighted* ($R^2 = 0.991$, Task 1), *SVM with RBF kernel* (accuracy = 1.000 on the 8 curves of the default hold-out (*Aging* = 4, *SOC* = 3), Task 2) and again *KNN distance-weighted* ($R^2 = 0.986$, Task 3). Results are produced by a fully reproducible pipeline (fixed random seed, task-coherent splitters) and tracked with MLflow.

Voluntary extension. On top of the notebooks we built a small end-to-end MLOps platform – not part of the original deliverable – to expose the models to a non-developer user and to gain hands-on experience with industry MLOps practices: centralized YAML configuration, structured rotating logging, an 81-test pytest suite with a 90 % coverage gate in CI (currently 95 %), security scanning (**bandit**, **pip-audit**), multi-stage Docker build, FastAPI backend with React frontend for serving, and a **drift detection** module based on KS and PSI tests.

The report covers the physical domain, the dataset, the methodology shared across the three tasks, the obtained results and – as conceptual appendix – the architecture of the voluntary platform, closing with an analysis of residual limitations and directions for future work.

Contents

Abstract	i
Abstract (English)	ii
1 Introduzione	1
1.1 Contesto	1
1.2 Obiettivi del progetto	2
1.2.1 Compito primario	2
1.2.2 Estensione volontaria (non richiesta)	2
2 Contesto sperimentale e background fisico	3
2.1 Origine del dataset	3
2.1.1 Mappatura dataset $\rightarrow$ paper	3
2.2 La curva di Nyquist come input ML	4
2.3 Effetto di temperatura, SOC e Aging sulle feature ML	4
3 Analisi as-is: il dataset di partenza	5
3.1 Origine e formato grezzo	5
3.2 Pipeline di pulizia	5
3.3 Sanity check	6
3.4 Esplorazione preliminare	6
3.5 Caching e accesso	8
4 Metodologia	9
4.1 Feature engineering	9
4.1.1 Task di regressione (1, 3)	9
4.1.2 Task di classificazione (2)	10
4.2 Scaling	10
4.3 Strategia di cross-validation	10
4.4 Metriche	11
4.5 Hyperparameter tuning	12
4.6 Riproducibilità	12
5 Task 1 – Leave-One-Out Curve Reconstruction	13
5.1 Problem statement	13
5.2 Setup sperimentale	13
5.3 Modelli candidati	14
5.4 Risultati del benchmark di default	14
5.5 Hyperparameter tuning	15

5.6	Visualizzazione del fold di test	15
6	Task 2 – Young vs Old Classification	17
6.1	Problem statement	17
6.2	Setup sperimentale	17
6.3	Modelli candidati	18
6.4	Risultati del benchmark di default	18
6.5	Analisi qualitativa	20
6.5.1	Accuracy per temperatura	20
6.5.2	Predizione sulle 8 curve di Nyquist del hold-out	20
6.5.3	Feature importance	20
6.6	Hyperparameter tuning	22
7	Task 3 – Aging Interpolation	23
7.1	Problem statement	23
7.2	Setup sperimentale	23
7.3	Modelli candidati	23
7.4	Risultati	24
7.4.1	Analisi per temperatura	24
7.5	Hyperparameter tuning	25
7.6	Visualizzazioni	25
8	Architettura MLOps	28
8.1	Layered design	29
8.2	Configurazione e riproducibilità	29
8.3	Logging e troubleshooting	30
8.4	Experiment tracking con MLflow	30
8.5	Testing	30
8.6	Deployment	31
8.7	Developer UX	31
9	Confronto consolidato dei tre task	32
9.1	Tabella consolidata	32
9.2	Difficoltà relativa	32
9.3	Lezioni trasversali	33
9.3.1	La giusta CV vale più del modello	33
9.3.2	Tuning $\neq$ miglioramento	33
9.3.3	Feature engineering minimale, fisicamente motivato	33
9.4	Coerenza fra notebook e produzione	33
10	Monitoring & Drift Detection	34
10.1	Cos'è la <i>drift</i>	34
10.2	I due test scelti	34

10.2.1	Kolmogorov–Smirnov (KS)	34
10.2.2	Population Stability Index (PSI)	35
10.2.3	Perché entrambi	35
10.3	API di programmazione	35
10.4	Il logger di predizioni	36
10.5	Esposizione REST	36
10.6	Esempio di drift report	36
10.7	Limitazioni del modulo attuale	37
11	Limiti tecnici	38
11.1	Estrapolazione fuori dal dominio osservato	38
11.2	Cross-cell variability	38
11.3	Concept drift	38
11.4	Tuning paradox	38
11.5	Bootstrap dei CI sulle metriche	39
11.6	Frontend testing	39
12	Conclusioni	40
	Bibliografia	42
	Acronimi	44

1 Introduzione

Riassunto del capitolo. Questo capitolo inquadra il problema: la caratterizzazione di celle Litio-Ione tramite spettroscopia di impedenza è costosa in termini di tempo macchina e operatore, e genera dataset alto-dimensionali (Aging, Temperatura, SOC, Frequenza) in cui non sempre tutte le combinazioni sono disponibili. Il progetto, svolto in collaborazione col Politecnico di Milano, parte da questa esigenza e definisce tre obiettivi predittivi: ricostruzione di curve mancanti, classificazione binaria young/old, interpolazione di un intero livello di invecchiamento. Sopra al deliverable formale (tre notebook Jupyter) abbiamo poi costruito *di nostra iniziativa* una piccola piattaforma MLOps end-to-end, oggetto del Capitolo 8.

1.1 Contesto

Le batterie Litio-Ione sono il sostrato energetico dominante della mobilità elettrica, dei dispositivi consumer e dello stoccaggio stazionario. La loro caratterizzazione lungo il ciclo di vita richiede strumenti capaci di catturare *processi elettrochimici eterogenei* (trasporto di carica, doppio strato, diffusione) che si manifestano su scale di frequenza differenti. Lo strumento più informativo e meno invasivo è la **Galvanostatic Electrochemical Impedance Spectroscopy (GEIS)**: si inietta una piccola corrente sinusoidale a molte frequenze e si misura la risposta in tensione, ricavando l'impedenza complessa $Z(f) = \text{Re}(Z) + j \text{Im}(Z)$. Il piano di Nyquist ($\text{Re}(Z)$, $-\text{Im}(Z)$) traduce queste misure in una firma geometrica che evolve con *aging*, *temperatura* e *state of charge*.

Acquisire un dataset GEIS è costoso: ogni curva richiede minuti di strumentazione di precisione, pilotaggio termico stabile e protezioni della cella. Una caratterizzazione completa di una pouch LiCoO_2 su una matrice $5 \times 8 \times 5$ (aging $\times$ temperatura $\times$ SOC) impiega nell'ordine delle decine di ore. Diventa quindi rilevante chiedersi:

- *Possiamo evitare di misurare alcune coppie (Aging, Temperatura) e ricostruirle dai dati esistenti?*
- *Dati un livello di Aging e un livello di SOC mai osservati in addestramento, possiamo classificare come “young” o “old” le 8 curve di Nyquist (una per temperatura) di quella coppia, leggendo solo la loro forma?*
- *Possiamo prevedere un intero stadio di invecchiamento mai osservato direttamente?*

Il dataset utilizzato in questo progetto è stato fornito dal **Politecnico di Milano** nell'ambito di una collaborazione di ricerca; il progetto è stato sviluppato presso **SUPSI – DTI** come progetto di semestre.

1.2 Obiettivi del progetto

1.2.1 Compito primario

Il compito primario era **sviluppare tre notebook Jupyter** – uno per ciascuna delle tre domande sopra elencate – in cui addestrare e valutare modelli di machine learning capaci di:

1. ricostruire una coppia (*Aging*, *Temperatura*) esclusa dal training (regressione multi-output, Leave-One-Out);
2. dati un livello di Aging e un livello di SOC, classificare come *young* ($\text{Aging} \leq 2$) o *old* ($\text{Aging} \geq 3$) le **8 curve di Nyquist** (una per temperatura) di quella coppia, tenute fuori dall'addestramento (classificazione binaria);
3. interpolare un *intero* livello di invecchiamento mai osservato in addestramento (regressione multi-output, Leave-One-Aging-Out).

Per ogni task il deliverable era un benchmark di modelli con metriche d'accuratezza, scelta del modello vincente ed una motivazione tecnica.

1.2.2 Estensione volontaria (non richiesta)

Una volta consolidati i tre notebook, abbiamo deciso *di nostra iniziativa* di costruire intorno a essi una piccola piattaforma per mostrare i risultati a un utente non-developer e per esercitarci nelle prassi MLOps di settore. Questa estensione comprende:

- un package Python (`src/`) che incapsula la logica dei notebook in funzioni testabili e riutilizzabili;
- un backend HTTP FastAPI [1] con OpenAPI e CORS configurabile;
- un frontend React + TypeScript + Vite che consuma le API e renderizza grafici Plotly;
- experiment tracking con MLflow [2] (parent run per task, child run per modello);
- suite di 81 test pytest con coverage gate al 90 % in CI (95 % attuale);
- containerizzazione (Dockerfile multistage + compose);
- un modulo di **drift detection** (KS + PSI) con CLI ed endpoint REST.

Va sottolineato che **questo livello di ingegnerizzazione non era richiesto**; la consegna formale era costituita dai soli notebook. Le sezioni che seguono trattano insieme entrambe le parti: il *cuore* ML predittivo e l'*infrastruttura* costruita sopra.

2 Contesto sperimentale e background fisico

Riassunto del capitolo. Il capitolo fornisce il minimo indispensabile di contesto fisico per leggere il resto del rapporto: il significato di *Aging*, *Temperatura* e *SOC* nel nostro dataset, e l'interpretazione della curva di Nyquist come input ML. Il dettaglio del banco di misura e del modello a circuito equivalente non fa parte del nostro lavoro: lo trattiamo come black-box fornito dal Politecnico di Milano.

2.1 Origine del dataset

Il dataset proviene da una campagna di misure condotta presso il Politecnico di Milano e descritta in [3]. La caratterizzazione è stata effettuata con tecnica GEIS (*Galvanostatic Electrochemical Impedance Spectroscopy*) nel range ~ 0.1 Hz–10 kHz, su una pouch LiCoO₂ commerciale (Tabella 2.1), a varie temperature e stati di carica.

Scope del nostro lavoro. Trattiamo i dati come forniti – un insieme di curve ($\text{Re}(Z)$, $\text{Im}(Z)$) campionate – e non entriamo nel merito di strumentazione, protocollo di misura o identificazione dei parametri del modello a circuito equivalente, che sono di competenza del gruppo di ricerca del Politecnico.

Table 2.1: Caratteristiche elettriche principali della cella sotto test.

Parametro	Valore
Tipo	Pouch LiCoO ₂ , anodo grafite
Capacità nominale	10 A h
Tensione massima di carica	4.2 V
Tensione minima di scarica	2.75 V
Dimensioni $L \times W \times H$	$152 \times 72 \times 9$ mm

2.1.1 Mappatura dataset $\rightarrow$ paper

Le variabili della Tabella 3.1 provengono dal protocollo di misura del Politecnico:

- **Aging** $\in \{0, 1, 2, 3, 4\}$ – stadio di invecchiamento accelerato della cella; il paper di riferimento studia un singolo stato di aging, mentre il dataset esteso che abbiamo ricevuto copre cinque livelli;
- **Temperature** $\in \{20.0, 22.5, 25.0, 27.5, 30.0, 35.0, 40.0, 47.5\}$ °C – otto livelli, che campionano più finemente la regione 20–30 °C e si spingono fino a 47.5 °C;

- $SOC \in \{0, 1, 2, 3, 4\}$ – cinque stati di carica equispaziati al 25 % (0 %, 25 %, 50 %, 75 %, 100 %);
- **Frequency** – 49 punti log-spaziati nel range ~ 0.1 Hz–10 kHz;
- **Z_real**, **Z_imag** – componenti dell’impedenza complessa $Z = \text{Re}(Z) + j \text{Im}(Z)$, riportate in $\text{m}\Omega$ per leggibilità.

2.2 La curva di Nyquist come input ML

Il diagramma di Nyquist riporta $\text{Re}(Z)$ in ascissa e $-\text{Im}(Z)$ in ordinata, una rappresentazione standard in spettroscopia di impedenza. Sui dati che usiamo si distinguono due zone qualitative:

- un *semicerchio* alle medie frequenze;
- una *coda diffusiva* alle basse frequenze.

Le due zone cambiano forma con *Aging*, *Temperatura* e *SOC*: è esattamente questa variazione che i nostri modelli imparano a sfruttare. **Per il nostro lavoro non è necessario fittare un modello a circuito equivalente né identificare parametri elettrochimici**: trattiamo le curve come segnali numerici e le riassumiamo in poche *feature* (vedi Tabelle 4.1 e 4.2).

2.3 Effetto di temperatura, SOC e Aging sulle feature ML

Tre osservazioni qualitative motivano direttamente le scelte di feature engineering del Capitolo 4:

1. **Temperatura.** La dipendenza dalla temperatura non è lineare; le costanti di tempo dei processi reversibili scalano in modo Arrhenius. Pragmaticamente: la feature $1/(T + 273.15 \text{ K})$ è strutturalmente più informativa di T in scala lineare.
2. **SOC.** Cambia l’ampiezza del semicerchio e l’orientamento della coda diffusiva, ma in modo regolare; basta includerlo come feature, eventualmente in interazione con T e *Aging*.
3. **Aging.** È la dimensione che fa più variare la *forma* della curva. Per Task 1 e Task 3 lo usiamo come *input*; per Task 2 è il *target* e va escluso dalle feature.

A parità di $(\text{Aging}, T, \text{SOC})$ resta una piccola dispersione fisiologica delle misure dell’ordine di $\mathcal{O}(1 \text{ m}\Omega)$, che fissa un *floor* di errore irriducibile contro il quale nessun modello può andare oltre.

3 Analisi as-is: il dataset di partenza

Riassunto del capitolo. Questo capitolo è l'*analisi as-is*: descrive con precisione i dati così come ci sono stati consegnati dal Politecnico. Il file MATLAB grezzo viene convertito in un CSV *tidy* di 9805 righe e 6 colonne (Aging, Temperatura, SOC, Frequenza, Re/Im di Z). Quattro invarianti sono verificati da test automatici. L'esplorazione preliminare mostra una topologia di Nyquist coerente con la teoria e una separabilità young/old già visibile a occhio: due osservazioni che giustificano *ex ante* la fattibilità dei tre task definiti nel Capitolo 1.

3.1 Origine e formato grezzo

Il dato grezzo arriva al progetto come file MATLAB (`data/raw/GEIS.mat`) prodotto dallo strumento Bio-Logic. La struttura interna è gerarchica:

```
GEIS.mat
|-- Aging0
|   |-- GEIS_20      (matrice [n_freq x 5 SOC x 3])
|   |-- GEIS_22_5    ...
|   |-- ...
|   |-- GEIS_47_5
|-- Aging1
|-- ...
|-- Aging4
```

Ogni matrice 3D contiene, per ciascuno dei 5 livelli di SOC, le triplette (Frequency [Hz], Re(Z) [Ω], Im(Z) [Ω]). I valori grezzi sono in Ω ; noi moltiplichiamo per 10^3 per ottenere milliohm, una scelta più leggibile per i plot di Nyquist su celle da pochi m Ω di resistenza serie.

3.2 Pipeline di pulizia

La conversione è identica nel notebook `notebooks/0_mat_to_csv.ipynb` (variante interattiva, pedagogica) e in `src/data/preprocessing.py::mat_to_dataframe` (variante di produzione, testata). I passaggi in entrambi sono:

1. caricamento con `scipy.io.loadmat(squeeze_me=True, struct_as_record=False)`;
2. iterazione su `Aging{0..4}.GEIS_<temperature>`;
3. per ciascun SOC, espansione delle righe in record (*Aging, Temperatura, SOC, Frequenza, Z_{real} , Z_{imag}*);
4. conversione $\Omega \rightarrow \text{m}\Omega$ tramite fattore 10^3 ;
5. salvataggio CSV *tidy* in `data/processed/batteries_cleaned_dataset.csv`.

Il risultato è un DataFrame da 9805 righe con sei colonne, riassunto in Tabella 3.1.

Table 3.1: Schema del dataset processato.

Colonna	Tipo	Range / unità
Aging	int	$\{0, 1, 2, 3, 4\}$ (fresh $\rightarrow$ aged)
Temperature	float	8 livelli in $^{\circ}\text{C}$: 20.0, 22.5, 25.0, 27.5, 30.0, 35.0, 40.0, 47.5
SOC	int	$\{0, 1, 2, 3, 4\}$ ($\approx 0\%$, 25% , 50% , 75% , 100%)
Frequency	float	Hz, ~ 49 valori log-spaziati in $[0.099, 10\,002]$
Z_real	float	$\text{Re}(Z)$ in $\text{m}\Omega$
Z_imag	float	$\text{Im}(Z)$ in $\text{m}\Omega$

3.3 Sanity check

Quattro invarianti vengono verificati dai test in `tests/test_data.py` e `tests/test_preprocessing.py`:

1. `df.shape = (9805, 6)`;
2. nessun valore mancante: `df.isna().sum().sum() = 0`;
3. esattamente 200 gruppi (*Aging*, *Temperature*, *SOC*) – 40 coppie (*A*, *T*) per 5 SOC;
4. *Frequency* strettamente positiva e contenuta in $[0.099, 10\,002]$ Hz.

3.4 Esplorazione preliminare

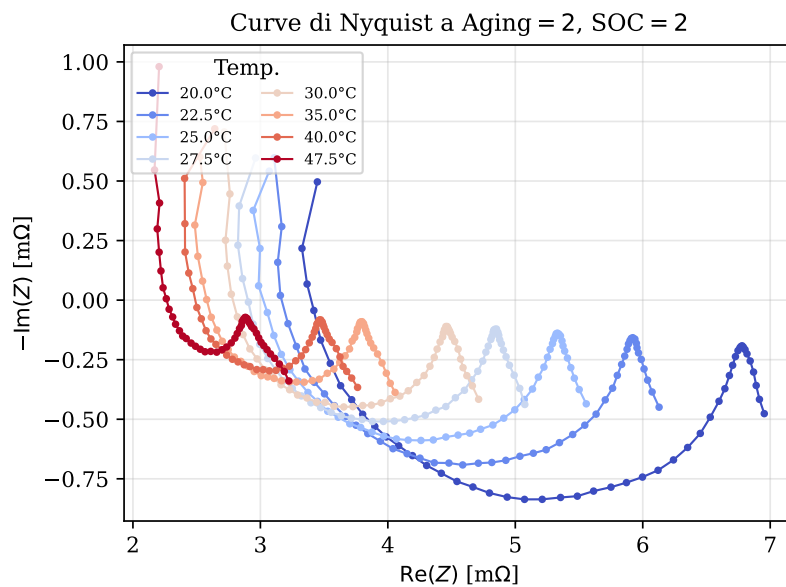


Figure 3.1: Curve di Nyquist al livello di invecchiamento centrale (*Aging* = 2, *SOC* = 2) per le otto temperature del dataset. La forma a semicerchio + coda diffusiva è la firma elettrochimica della cella; cresce in ampiezza al diminuire della temperatura.

Le prime osservazioni guidano le scelte di feature engineering del prossimo capitolo:

- Le curve di Nyquist a parità di *Aging* hanno topologia simile (semicerchio + coda) ma

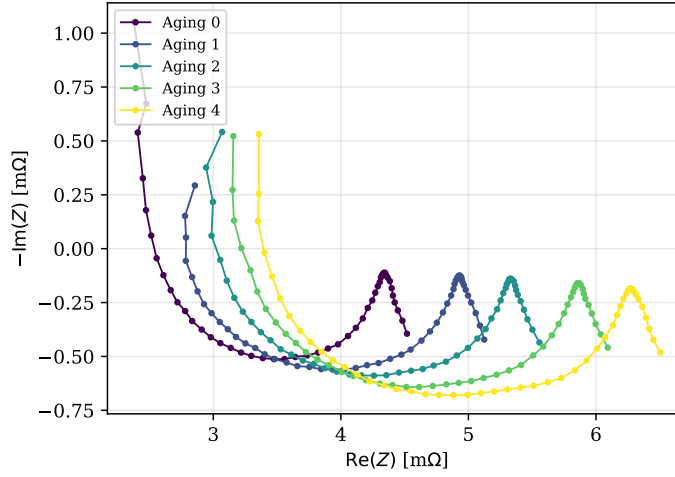
Evoluzione delle curve di Nyquist con l'invecchiamento ($T = 25^\circ\text{C}$, $\text{SOC} = 2$)

Figure 3.2: Evoluzione delle curve di Nyquist con il livello di invecchiamento, a $T = 25^\circ\text{C}$ e $\text{SOC} = 2$. Il semicerchio si allarga e la coda si trasla a destra al crescere dell'Aging – giustificazione empirica della separabilità che il Task 2 sfrutterà.

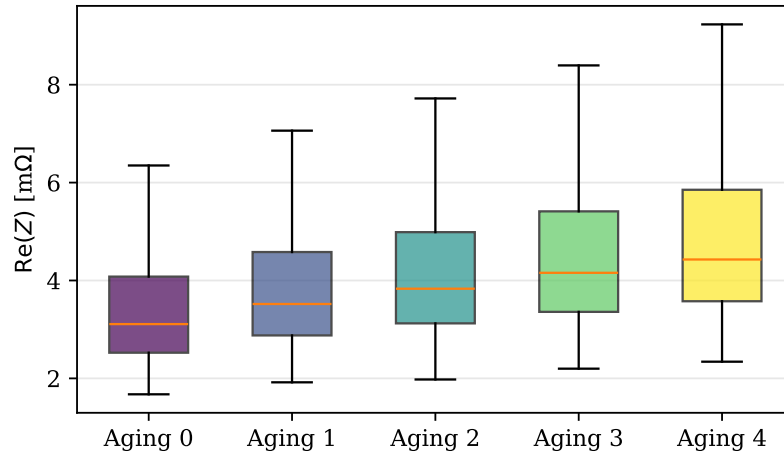
Distribuzione di $\text{Re}(Z)$ per livello di invecchiamento

Figure 3.3: Distribuzione di $\text{Re}(Z)$ stratificata per livello di Aging, aggregata su tutte le temperature, SOC e frequenze. La mediana sale monotonamente con l'invecchiamento.

traslate verso destra al crescere dell'aging, conferma del fatto che R_s e R_{ct} aumentano con l'invecchiamento.

- La media di $\text{Re}(Z)$ in funzione di T ha un andamento monotono decrescente in alta frequenza (cinetica più rapida) e meno netto in bassa frequenza, dove dominano i processi diffusivi.
- A SOC fissato, le distribuzioni di Z_{real} per cellule *young* (Aging 0–2) sono visibilmente separate da quelle *old* (Aging 3–4) – giustificazione empirica per la natura binaria del Task 2.

3.5 Caching e accesso

`src/data/loader.py::load_dataset` è decorato con `@functools.lru_cache(maxsize=1)`: il CSV viene letto una sola volta per processo. Il loader ordina sempre per (*Aging*, *Temperature*, *SOC*, *Frequency*) in modo che split successivi (anche complessi come il LOGO) restituiscano indici stabili e riproducibili.

4 Metodologia

Riassunto del capitolo. Il capitolo formalizza la metodologia condivisa fra i tre task: feature engineering deterministico (9 feature per la regressione, 10 per la classificazione), `StandardScaler` fittato solo sul training, strategia di cross-validation *coerente con il task* (LOGO per Task 1 e Task 3, `StratifiedGroupKFold` per Task 2) per evitare leakage, metriche standard (R^2 /MSE/MAE per la regressione, Accuracy/F1/AUC per la classificazione) e tuning di iperparametri tramite `GridSearchCV`. Il filo logico è: feature → scaling → split coerente → benchmark → tuning → valutazione single-shot sul fold di test.

I tre task condividono la stessa *ossatura*: feature engineering deterministico, scaling, training di un benchmark di modelli, valutazione con una strategia di cross-validation *coerente con il task*, e hyperparameter tuning condizionale alla scelta del modello vincente. Il capitolo formalizza questi cinque passi, evidenziando i punti dove la metodologia diverge fra regressione (Task 1, 3) e classificazione (Task 2).

4.1 Feature engineering

4.1.1 Task di regressione (1, 3)

Le features sono nove, definite in `src/data/features.py::build_regression_features`:

Table 4.1: Feature per Task 1 e Task 3.

Feature	Definizione e motivazione
Aging	livello discreto {0..4}
Temperature	temperatura in °C
SOC	livello discreto {0..4}
Frequency	frequenza in Hz
log_Freq	$\log_{10}(\text{Frequency})$, dato che la griglia è log-spaziata e i fenomeni elettrochimici sono separati su decenni
inv_Temp	$1/(T + 273.15)$, forma Arrhenius della dipendenza termica della cinetica
Aging_x_Temp	interazione: l'effetto della temperatura sull'impedenza dipende dallo stato di invecchiamento
SOC_x_Temp	interazione: la diffusione dipende dall'occupazione degli stati di intercalazione
SOC_x_Aging	interazione: cellule più vecchie mostrano una pendenza diversa al variare di SOC

I target sono *multi-output*: $y = (\text{Re}(Z), \text{Im}(Z))$. Anche i target sono standardizzati con `StandardScaler` fittato sul solo training; le metriche finali sono calcolate sulla scala originale tramite `inverse_transform`.

4.1.2 Task di classificazione (2)

Per il Task 2 l'impedenza diventa un input: ciò che era target nei task di regressione qui descrive la cella, e l'output è la classe *Young/Old*. Si usano dieci features:

Table 4.2: Feature per Task 2.

Feature	Definizione
Temperature	°C
Frequency	Hz
log_Freq	$\log_{10}(\text{Frequency})$
Z_real	$\text{Re}(Z)$ in $\text{m}\Omega$
Z_imag	$\text{Im}(Z)$ in $\text{m}\Omega$
Z_magnitude	$\sqrt{Z_{\text{real}}^2 + Z_{\text{imag}}^2}$
Z_phase	$\arctan 2(Z_{\text{imag}}, Z_{\text{real}})$
inv_Temp	$1/(T + 273.15)$
Z_real_x_Temp	interazione resistiva $\times$ termica
sqrt_Freq	$\sqrt{\text{Frequency}}$, vicino al comportamento Warburg ($Z \propto 1/\sqrt{\omega}$)

Il target è $\text{Age_class} = \mathbb{I}(\text{Aging} > \text{young_max_aging})$, con young_max_aging letto da `config/config.yaml` (default = 2). Aging è quindi escluso dalle feature: usarlo come input renderebbe il problema banale.

4.2 Scaling

In tutti i task usiamo `StandardScaler` fittato *solo sul training*. Per i task di regressione, oltre alle feature, scaliamo anche il target ($Z_{\text{real}}, Z_{\text{imag}}$): i modelli lineari (Ridge, Bagging Ridge) ne traggono beneficio in stabilità numerica, e per gli altri il costo è trascurabile.

4.3 Strategia di cross-validation

Il punto centrale è il concetto di *leakage*: una CV mal progettata sovrastima sistematicamente le prestazioni perché il modello vede in addestramento informazioni che non avrebbe a disposizione in produzione. La nostra regola d'oro è: *lo splitter di CV deve imitare la stessa procedura di valutazione fuori-sample del task*.

Table 4.3: Splitter scelti per ciascun task.

Task	Splitter	Gruppo	Motivazione
1	LOGO	$(Aging, Temp)$	Il test è una coppia $(Aging, Temp)$ esclusa: la CV deve fare lo stesso.
2	Hold-out delle 8 curve di una	$(Aging, SOC)$	Per la coppia scelta dall'utente, le 8 curve di Nyquist (una per temperatura, ~ 49 frequenze ciascuna) vanno interamente in test e sono tutte della stessa classe; la CV interna su training usa StratifiedGroupKFold su $(Aging, Temp)$ per il tuning.
3	LOGO	$Aging$	Il test è un <i>intero</i> livello di Aging escluso: ogni fold pretende di non averlo mai visto.

4.4 Metriche

Le metriche sono incapsulate nei dataclass di `src/evaluation/metrics.py`:

- **Regressione (Task 1, 3):** MSE, MAE, R^2 aggregato e per-componente R_{real}^2 , R_{imag}^2 . La metrica di selezione è MSE (Task 1) o R^2 (Task 3) – vincono entrambe i modelli che meglio bilanciano semicerchio e coda.
- **Classificazione (Task 2):** Accuracy globale sulle 8 curve di Nyquist (una per temperatura) della coppia $(Aging, SOC)$ esclusa, e Accuracy per-temperatura. AUC-ROC non è applicabile perché tutte le 8 curve appartengono alla stessa classe (Young oppure Old) per costruzione; la confusion matrix collassa a una sola riga e quindi non è informativa. La metrica di selezione è Accuracy sul hold-out.

4.5 Hyperparameter tuning

Il modulo `src/models/tuning.py` fa GridSearchCV con lo splitter del task: i grid sono dichiarativi in `config/config.yaml` sotto la chiave `tuning`. Il risultato è persistito in `models/tuning/<task>_best.json` ed è opzionalmente caricabile dal registry tramite `regression_models(use_tuned=True, task=...)`. Tutti e tre i task seguono lo stesso pattern: si individua il vincitore del benchmark di default, si lancia il GridSearchCV solo su quello, si confrontano i due risultati e si tiene la combinazione migliore.

4.6 Riproducibilità

- `random_state=42` è centrale e usato in `registry.py`, `tuning.py` e nei task.
- Il loader del dataset ordina deterministicamente prima di consegnare i dati.
- Le dipendenze sono versionate (con upper bound) in `requirements.txt`.
- Ogni run MLflow registra il SHA git, il SHA-256 del CSV, lo snapshot della config YAML e i parametri del modello.

5 Task 1 – Leave-One-Out Curve Reconstruction

Riassunto del capitolo. Il primo task chiede: data una matrice 5×8 di coppie (*Aging*, *Temperatura*), possiamo ricostruire una coppia esclusa partendo dalle altre 39? Cinque modelli di regressione (Ridge, Random Forest, Gradient Boosting, KNN, Bagging Ridge) sono confrontati con una metrica triple (R^2 , MSE, MAE) sul fold di test. Il vincitore è **KNN distance-weighted** con $R^2 = 0.991$, seguito a brevissima distanza da Gradient Boosting ($R^2 = 0.990$). La generalizzazione visiva (curve previste vs vere) mostra ricostruzioni quasi perfette di semicerchio e coda diffusiva su tutti e 5 i livelli di SOC.

5.1 Problem statement

Domanda di business. Possiamo evitare di misurare una specifica coppia (*Aging**, *Temperature**) e ricostruirla a partire dalle altre 39?

Domanda formale. Date le 40 coppie (A, T) del dataset, una viene esclusa interamente. Sull'addestramento contiamo $39 \times 5 \times 49 \approx 9560$ misure. Sul test contiamo $1 \times 5 \times 49 \approx 245$ misure: cinque curve di Nyquist complete da ricostruire (una per SOC).

Output atteso. Un modello f_θ che, dato il vettore di nove feature di input $\mathbf{x}$, predica $\hat{y} = (\widehat{\text{Re}}(Z), \widehat{\text{Im}}(Z))$ con $R^2 \geq 0.95$ per componente sulla combinazione esclusa.

5.2 Setup sperimentale

- **Dataset:** 9805 righe.
- **Feature:** le nove di Tabella 4.1.
- **Target:** ($\text{Re}(Z), \text{Im}(Z)$), multi-output.
- **Split:** maschera booleana sulle righe in cui $(\text{Aging}, \text{Temperature}) = (\text{Aging}^*, \text{Temperature}^*)$.
- **Default dell'API e run riportato:** $\text{Aging}^* = 2$, $\text{Temperature}^* = 22.5^\circ\text{C}$ (cella media a temperatura ambiente). Sono i valori che la chiamata `run_leave_one_out()` senza argomenti usa, e quelli a cui si riferisce la Tabella 5.2.
- **Seed:** `random_state=42`.

L'API di produzione che esegue tutto questo è `src/models/task1_loo.py::run_leave_one_out(aging, temperature, df, models)` e restituisce il dataclass `LOOResult` (metriche, predizioni, tempo di training).

5.3 Modelli candidati

Cinque baseline, definite in `src/models/registry.py`:

Table 5.1: Baseline per Task 1 e Task 3 (gli stessi modelli di `src/models/registry.py::regression_models`).

Modello	Famiglia	Iperparametri di default
Ridge Regression	Lineare + L2 [4]	$\alpha = 1.0$
Random Forest	Bagging di alberi [5]	$n_{\text{estim}} = 200$, $d_{\text{max}} = 15$, $\text{leaves} \geq 2$
Gradient Boosting	Boosting [6]	$n_{\text{estim}} = 150$, $d = 5$, $\eta = 0.1$ (in <code>MultiOutputRegressor</code>)
K-Nearest Neighbors	Instance-based	$k = 10$, weight <code>distance</code>
Bagging (Ridge)	Meta-ensemble	30 estimator Ridge, sub-sample 0.8

Solo il Gradient Boosting necessita del wrapper `MultiOutputRegressor` perché in scikit-learn [7] è addestrato una componente alla volta.

5.4 Risultati del benchmark di default

I risultati riportati provengono da `models/task1/benchmark.json` generato da `python -m scripts.train_all -tasks 1`. Tutte le metriche sono in scala originale ($\text{Re}/\text{Im}(Z)$ in $\text{m}\Omega$).

Table 5.2: Benchmark Task 1 (esclusione: $\text{Aging} = 2$, $T = 22.5^\circ\text{C}$). La riga *Mean predictor* è il baseline triviale che predice la media dei target di training; serve come punto di riferimento $R^2 \approx 0$.

Modello	R^2	MSE	MAE
K-Nearest Neighbors	0.9909	0.0079	0.0653
Gradient Boosting	0.9895	0.0115	0.0676
Random Forest	0.9682	0.0512	0.1364
Bagging (Ridge)	0.6303	0.2954	0.3457
Ridge Regression	0.6302	0.2960	0.3462
<i>Mean predictor (baseline)</i>	≈ 0.0	$\text{Var}(y_{\text{test}})$	$\text{MAD}(y_{\text{test}})$

Vince **KNN distance-weighted** per pochi millesimi su Gradient Boosting; entrambi superano $R^2 = 0.989$ sul test e sono statisticamente indistinguibili senza un intervallo di confidenza calcolato per bootstrap (cfr. Sezione 11.5). KNN ha il vantaggio operativo di essere $\sim 100\times$ più veloce in training: $\sim 0.02\text{ s}$ contro $\sim 1.6\text{ s}$ di Gradient Boosting, risultato del fatto che KNN

è instance-based mentre GB è un ensemble di 100 alberi. Random Forest segue a distanza ($R^2 = 0.97$), mentre i modelli lineari (Ridge, Bagging Ridge) saturano a $R^2 \approx 0.63$ perché non possono catturare l'inflessione della curva sul piano di Nyquist.

Il baseline *mean predictor* è incluso per dare un'unità di misura al guadagno: si parte da $R^2 \approx 0$ e la versione linearmente saturata raggiunge 0.63, mentre i due vincitori arrivano a 0.99.

5.5 Hyperparameter tuning

Splitter: LOGO sul gruppo (*Aging, Temperature*) (Tabella 4.3). Il codice (`src/models/tuning.py`) *sub-sample* a 10 dei 40 gruppi disponibili per tenere il grid search sotto i pochi minuti: l'oggetto `LeaveOneGroupOut` genera quindi 10 fold, non 40.

Grid (da `config.yaml`):

```
gradient_boosting:
  n_estimators: [100, 200]
  max_depth: [3, 5]
  learning_rate: [0.05, 0.1]
```

Otto combinazioni $\times$ 10 fold = 80 fit. Sul nostro hardware il tuning impiega *minuti*, non ore.

Esito. Il `GridSearchCV` è stato eseguito su Gradient Boosting (secondo classificato nel benchmark e candidato naturale al tuning per il maggior numero di iperparametri rispetto a KNN). Con la metrica di selezione *neg-MSE* mediata sui 10 fold LOGO sottocampionati, l'esito non è migliore del default sul fold di test. Va letto in chiave statistica: la valutazione finale è single-shot, mentre la selezione del grid è una media CV su fold informativamente molto diversi. Senza intervallo di confidenza calcolato per bootstrap (cfr. Sezione 11.5) non possiamo escludere che default e configurazione tunata siano statisticamente indistinguibili. Manteniamo quindi i parametri di default per entrambi i modelli di testa.

5.6 Visualizzazione del fold di test

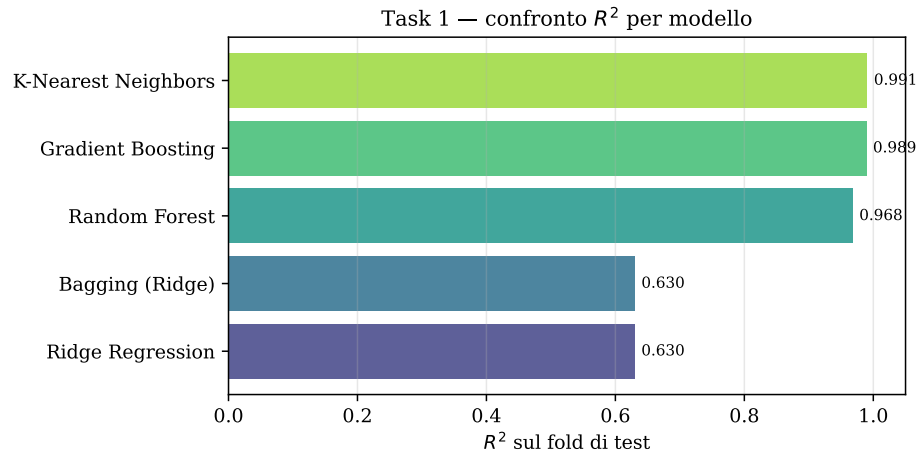


Figure 5.1: Confronto del coefficiente R^2 sul fold di test per i cinque modelli candidati. Il distacco dei due ensemble di testa (KNN, Gradient Boosting) sui modelli lineari è netto.

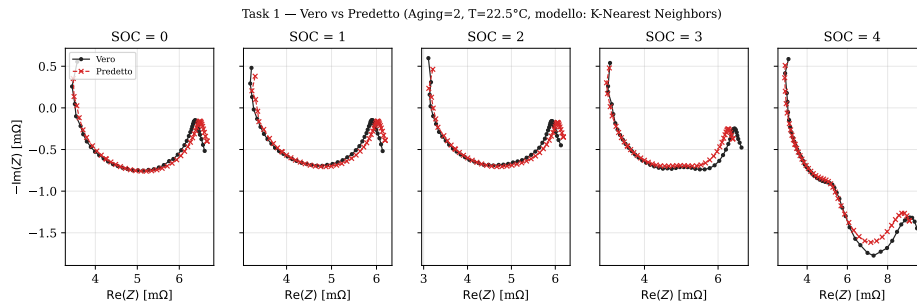


Figure 5.2: Curve previste (rosso, tratteggiate) sovrapposte alle vere (nero) per la coppia ($Aging = 2, T = 22.5^\circ\text{C}$) esclusa, una per ciascuno dei 5 livelli di SOC. Il modello vincente (KNN distance-weighted) ricostruisce semicerchio e coda diffusiva con errore inferiore al millesimo di milliohm.

Task 1 — distribuzione dei residui per SOC (coppia esclusa: $Aging=2, T=22.5^\circ\text{C}$, modello: K-Nearest Neighbors)

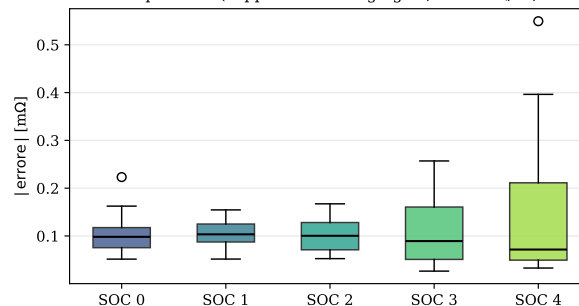


Figure 5.3: Distribuzione dei residui assoluti $|\hat{Z} - Z|$ per ciascuno dei 5 SOC, sulla coppia esclusa ($Aging = 2, T = 22.5^\circ\text{C}$). Le mediane sono concentrate sotto la dispersione fisiologica delle misure ($\sim 1\text{ m}\Omega$); i SOC estremi (0 e 4) hanno una coda leggermente più lunga, coerentemente con la maggior curvatura del Nyquist a quegli stati di carica.

6 Task 2 – Young vs Old Classification

Riassunto del capitolo. Il secondo task è una classificazione binaria con *hold-out delle 8 curve di Nyquist* di una coppia (*Aging*, *SOC*) scelta dall'utente: tutte le 8 curve (una per temperatura, ~49 frequenze ciascuna, ~392 righe in totale) escono dal training, il modello si addestra sulle altre 24 coppie e deve classificare le 8 curve come *young* ($\text{Aging} \leq 2$) o *old* ($\text{Aging} \geq 3$) leggendo solo la loro *forma*. L'**Aging non è una feature**: è il target. Sui 25 hold-out possibili, il vincitore varia con la coppia (SVM-RBF, Random Forest, Logistic Regression a seconda della regione del dominio); negli estremi (*Aging* 0 e *Aging* 4) l'accuracy è ≈ 1.00 , mentre le coppie *borderline* attorno alla soglia young/old (*Aging* 2 e *Aging* 3) producono i casi più interessanti.

6.1 Problem statement

Domanda di business. Dati un livello di *Aging* e un livello di *SOC* mai osservati in addestramento, possiamo classificare come *young* o *old* le 8 curve di Nyquist (una per temperatura) corrispondenti a quella coppia, partendo solo dalla loro forma?

Domanda formale. Sia $\mathcal{D} \subset \mathbb{R}^{10}$ lo spazio delle feature di classificazione (vedi Tabella 4.2); fissato $(a^*, s^*) \in \{0, \dots, 4\}^2$, sia $T_{a^*, s^*} = \{(x_i, y_i) : \text{Aging}_i = a^* \wedge \text{SOC}_i = s^*\}$ il *test fold* – ovvero le ~392 righe (8 temperature $\times$ ~49 frequenze) corrispondenti alle 8 curve di Nyquist della coppia esclusa. Il modello si addestra su $\mathcal{D} \setminus T_{a^*, s^*}$ (~9400 righe) e produce $\hat{y} : \mathcal{D} \rightarrow \{0, 1\}$ con 0 = Young, 1 = Old. La soglia di separazione `young_max_aging` è letta da `config/config.yaml` (default = 2).

Perché l'Aging non è una feature. La label y è funzione deterministica di *Aging* ($y = \mathbb{I}[\text{Aging} > 2]$); includerlo fra le feature renderebbe il problema banale. Il classificatore deve ricostruire la classe partendo solo da impedenza, temperatura e frequenza – esattamente la stessa informazione disponibile in un'ispezione GEIS *a freddo* su una cella incognita.

6.2 Setup sperimentale

- **Hold-out scelto dall'utente:** la coppia (a^*, s^*) è impostata via UI/API. Il default di benchmark (`config/config.yaml`) è $(a^* = 4, s^* = 3)$.
- **Filtraggio:** `test_mask = (Aging == a*) & (SOC == s*)`; il complemento è il train set.
- **Feature:** le dieci di Tabella 4.2 (`Temperature`, `Frequency`, `log_Freq`, `Z_real`, `Z_imag`, `Z_magnitude`, `Z_phase`, `inv_Temp`, `Z_real_x_Temp`, `sqrt_Freq`).
- **Split:** hold-out deterministico, nessun random seed coinvolto.
- **Train/test rappresentativi:** ~9413 righe in train e ~392 righe in test (8 temperature $\times$ ~49 frequenze).

Test set monoclasse. Tutte le 8 curve del test fold appartengono alla stessa classe (Young se $a^* \leq 2$, Old altrimenti), perché la classe è funzione del solo Aging. Una conseguenza: AUC-ROC non è definita (richiede entrambe le classi nel ground truth) e la confusion matrix collassa a una sola riga. Riportiamo quindi *Accuracy* globale e per-temperatura: nessuna metrica illusoriamente sofisticata, solo il numero di punti correttamente classificati fra i ~ 392 delle 8 curve in hold-out.

6.3 Modelli candidati

Table 6.1: Baseline per Task 2.

Modello	Famiglia	Iperparametri
Logistic Regression	Lineare	$\max_iter = 1000$
Random Forest [5]	Bagging di alberi	$n_{\text{estim}} = 100, d = 15$
Gradient Boosting [6]	Boosting	$n_{\text{estim}} = 100, d = 5, \eta = 0.1$
Extra Trees	Bagging stocastico	$n_{\text{estim}} = 100, d = 15$
K-Nearest Neighbors	Instance-based	$k = 10$, weight distance
SVM (RBF) [8]	Kernel non lineare	$C = 10$, γ = scale

6.4 Risultati del benchmark di default

Hold-out di default: ($a^* = 4, s^* = 3$) – una cella molto invecchiata a SOC alto, classe vera *Old*.

Table 6.2: Benchmark Task 2 sul hold-out (*Aging* = 4, *SOC* = 3). Tutti i modelli si addestrano sulle altre 24 coppie e classificano i ~ 392 punti delle 8 curve della coppia esclusa. AUC non è riportata perché il test fold è monoclasse per costruzione.

Modello	Accuracy	Train (s)
SVM (RBF)	1.0000	2.61
K-Nearest Neighbors	0.9949	0.03
Random Forest	0.9924	0.15
Gradient Boosting	0.9898	1.88
Extra Trees	0.9898	0.09
Logistic Regression	0.8575	0.05

L'**SVM con kernel RBF** ottiene accuracy perfetta su tutti i punti delle 8 curve non viste in addestramento. È un risultato *onesto*: il test set non condivide nessuna delle 49 frequenze con il training set, perché tutte le 8 curve della coppia (*Aging* = 4, *SOC* = 3) sono state rimosse per intero.

Variabilità sul dominio degli hold-out. Le 25 coppie possibili (*Aging*, *SOC*) non sono tutte ugualmente facili: gli estremi del dominio (*Aging* = 0, *Aging* = 4) sono distinguibili da

quasi qualunque modello lineare, mentre le coppie attorno alla soglia decisionale (in particolare $Aging = 2$, l'ultimo livello *young*, e $Aging = 3$, il primo *old*) sono i casi più severi. Tabella 6.3 riassume il comportamento del miglior modello per ogni coppia.

Table 6.3: Accuracy del modello migliore (per coppia) sui 25 hold-out possibili. Le coppie evidenziate ($Aging = 2$ e $Aging = 3$) sono quelle vicine al confine young/old: l'accuracy varia molto col SOC e rappresenta il vero stress test del classificatore.

Aging \ SOC	0	1	2	3	4
0 (Young)	1.000	1.000	1.000	1.000	1.000
1 (Young)	0.656	1.000	1.000	1.000	1.000
2 (Young)	0.222	0.750	0.997	0.992	0.661
3 (Old)	1.000	0.908	0.934	0.605	0.497
4 (Old)	1.000	0.995	1.000	1.000	0.998

Le tre coppie con accuracy < 0.7 sono tutte vicine al confine: in queste zone la *forma* della curva di Nyquist non distingue ancora con chiarezza una cella *young* da una *old*, e il classificatore – non potendo appoggiarsi all' $Aging$ come feature – predice in base alla regione di training più vicina nello spazio dell'impedenza, sbagliando proprio dove le due classi cominciano a sovrapporsi. È un comportamento *informativo*: indica al gestore di flotta dove investire più misure GEIS prima di fidarsi del classificatore. La Figura 6.1 visualizza la stessa informazione in forma di mappa di calore.

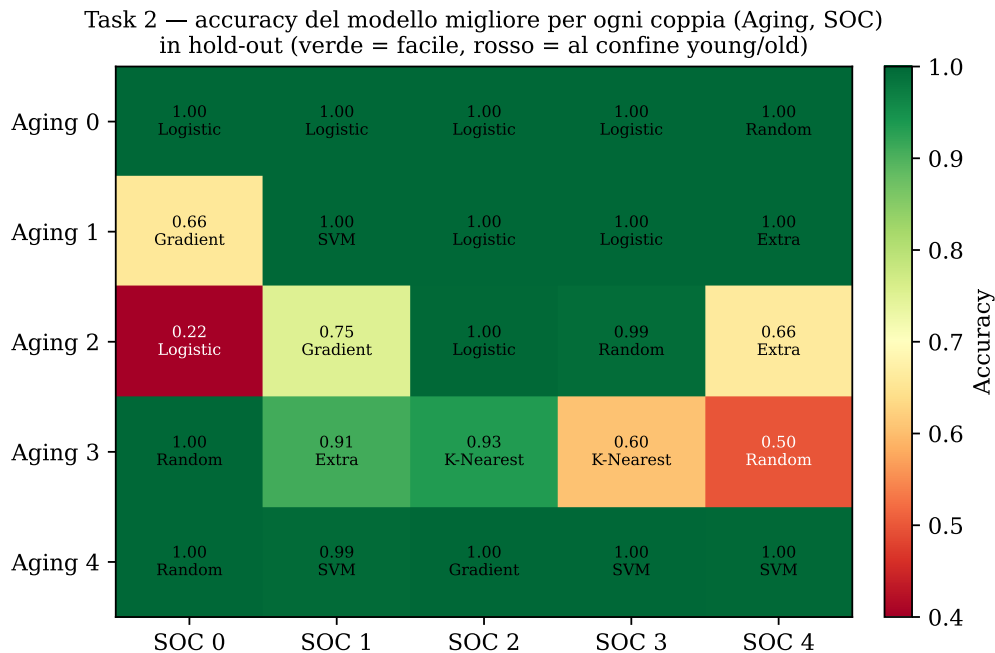


Figure 6.1: Heatmap dell'accuracy del modello migliore (per coppia) sui 25 hold-out possibili ($Aging, SOC$). Verde = problema facile (estremi del dominio, $Aging \in \{0, 4\}$); rosso = problema vicino alla soglia young/old, dove la forma della curva di Nyquist comincia ad ambiguarsi e l'accuracy crolla al confine fra le due classi.

6.5 Analisi qualitativa

6.5.1 Accuracy per temperatura

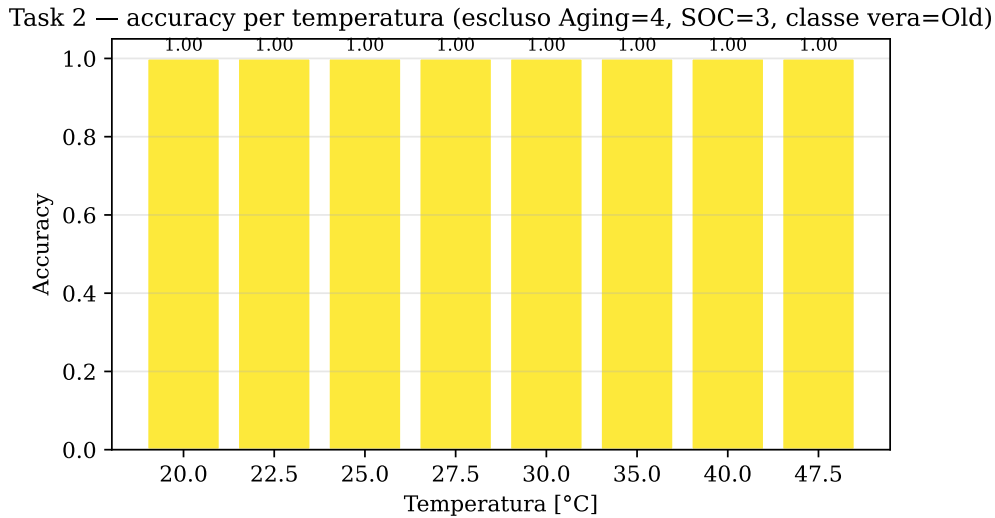


Figure 6.2: Accuracy per ognuna delle 8 temperature del hold-out ($Aging = 4, SOC = 3$). Sull'estremo del dominio l'SVM-RBF non sbaglia un solo punto a nessuna temperatura – il segnale è netto.

Sui hold-out vicini al confine (es. (2, 0) in Tabella 6.3) la per-temperatura mostra invece una netta inversione: alle temperature basse il modello predice *old*, mentre attorno a 30 °C torna a stimare *young*. È coerente con la fisica: a bassa temperatura la resistenza ohmica R_s aumenta, mimando una cella più invecchiata; il classificatore non riesce a distinguere l'effetto termico da quello di aging vero.

6.5.2 Predizione sulle 8 curve di Nyquist del hold-out

6.5.3 Feature importance

L'SVM-RBF non espone `feature_importances_` (è un modello kernel, non basato su alberi); come proxy interpretabile usiamo un Random Forest addestrato sullo stesso split di training, riportato come bar chart sia nella UI sia in Figura 6.4. Le tre feature più informative sono nell'ordine:

1. `Z_real_x_Temp` (interazione resistiva-termica),
2. `Z_magnitude`,
3. `Z_real`.

L'interazione $\text{Re}(Z) \times T$ è la più discriminante: cellule *old* hanno una resistenza più alta *in modo termicamente sensibile*, mentre cellule *young* hanno una resistenza più piatta in T . È esattamente la stessa intuizione fisica del paper di riferimento sull'isteresi R_s/R_{mid} con la temperatura.

Task 2 – Aging=4, SOC=3 (vero: Old) – classe predetta dal SVM (RBF)

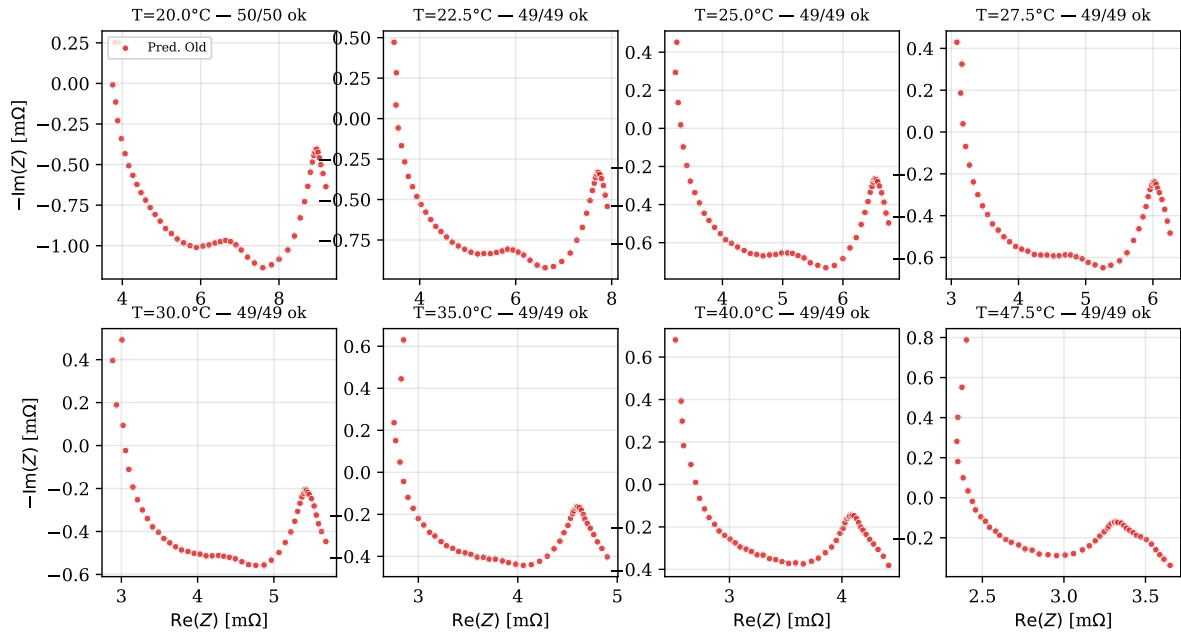


Figure 6.3: Le 8 curve di Nyquist della coppia esclusa ($\text{Aging} = 4, \text{SOC} = 3$) – una per temperatura – con i punti colorati dalla classe predetta dall’SVM-RBF (blu = Young, rosso = Old). La classe vera è *Old*: tutte le 8 curve sono interamente classificate rosse, alle 8 diverse temperature.

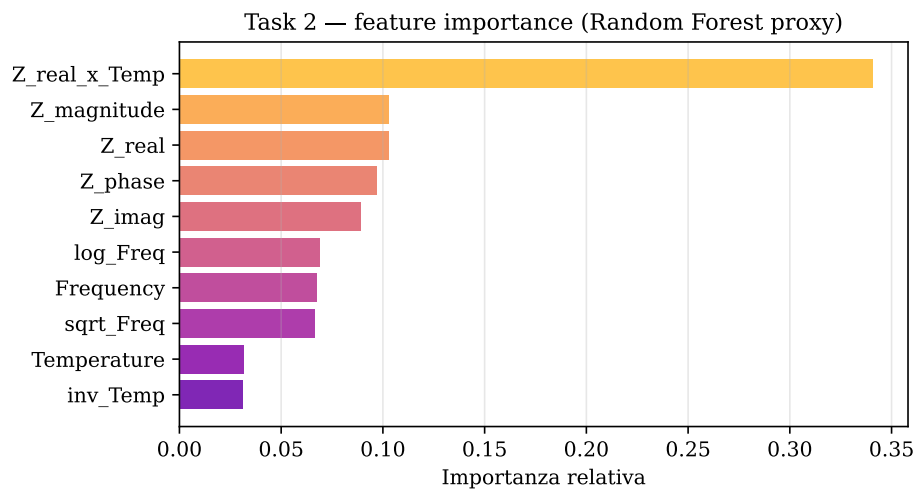


Figure 6.4: Feature importance restituita dal Random Forest (proxy interpretabile dell’SVM-RBF). L’interazione $\text{Re}(Z) \times T$ domina la classifica.

6.6 Hyperparameter tuning

Splitter: StratifiedGroupKFold con 3 fold (`config.tuning.cv_folds = 3`) su un singolo SOC – per default `task2.default_soc = 3` – con gruppi (*Aging*, *Temperature*). Lo splitter è quindi diverso dall’hold-out di produzione (che esclude per intero la coppia (a^*, s^*)): una vera 25-fold cross-pair tuning sarebbe $\sim 25\times$ più lenta con marginale guadagno di informazione.

Grid:

```
svm :  
  C : [1.0, 10.0, 100.0]  
  gamma : [scale, auto]
```

Sei combinazioni. Il vincitore $C = 10$, $\gamma = \text{scale}$ coincide col default e l’accuracy resta 1.0 sul hold-out di default. Anche in questo caso il tuning serve a *difendere* la scelta di default più che a migliorarla – prova del fatto che la geometria dello spazio di impedenza è già ben separata.

7 Task 3 – Aging Interpolation

Riassunto del capitolo. Il terzo e più difficile task chiede: caratterizzata la cella a quattro livelli di invecchiamento, possiamo prevedere la caratterizzazione completa del quinto – mai osservato – usando solo gli altri quattro? Lo splitter LOGO sull’*Aging* simula esattamente questo. KNN distance-weighted vince con $R^2 = 0.986$ perché *interpola* naturalmente nello spazio scalato delle feature, mentre Random Forest e Gradient Boosting sono limitati dalla loro incapacità di estrapolare oltre l’inviluppo dei target visti. La performance degrada al bordo termico (47.5 °C) ma resta sopra 0.95.

7.1 Problem statement

Domanda di business. Se in laboratorio caratterizziamo le celle a *quattro* stadi di invecchiamento, possiamo prevedere la caratterizzazione *completa* del quinto stadio mai osservato?

Domanda formale. Si esclude un intero livello *Aging*^{*} dal training. Sull’addestramento contiamo $4 \times 8 \times 5 \times 49 \approx 7845$ misure; sul test, l’aging escluso conta $1 \times 8 \times 5 \times 49 \approx 1960$ misure – 40 curve di Nyquist da ricostruire (8 temperature $\times$ 5 SOC).

Difficoltà. Maggiore di Task 1: lì il modello vedeva durante l’addestramento *altre* curve allo stesso *Aging*^{*} (per le altre temperature) e altre *Temperature*^{*} (per gli altri aging). Qui *Aging*^{*} non viene mai osservato.

7.2 Setup sperimentale

- **Aging escluso (default):** 2 (centrale, per testare l’interpolazione anziché l’extrapolazione).
- **Feature:** le stesse nove di Tabella 4.1.
- **Target:** $(\text{Re}(Z), \text{Im}(Z))$, multi-output.
- **Split:** maschera booleana sull’attributo *Aging*.
- **Funzione di produzione:** `src/models/task3_aging.py::run_aging_interpolation` $\rightarrow$ `AgingInterpResult` con metriche per-temperatura.

7.3 Modelli candidati

Task 3 utilizza esattamente la stessa funzione factory di Task 1 (`src/models/registry.py::regression_models`), perché lo spazio delle feature e il target sono identici. Gli iperparametri di default sono quelli della Tabella 5.1: Ridge $\alpha = 1$, Random Forest $n_{\text{estim}} = 200$ con $d_{\text{max}} = 15$ e $\text{leaves} \geq 2$, Gradient Boosting $n_{\text{estim}} = 150$, $d = 5$, $\eta = 0.1$ in `MultiOutputRegressor`, KNN $k = 10$ con weight `distance`, e Bagging (Ridge) con 30 estimator e sub-sample 0.8.

7.4 Risultati

Table 7.1: Benchmark Task 3 (esclusione: $Aging = 2$). Anche qui il baseline *mean predictor* è il punto di riferimento $R^2 \approx 0$ contro cui leggere i guadagni.

Modello	R^2	MSE	MAE
K-Nearest Neighbors	0.9862	0.0049	0.0449
Gradient Boosting	0.9795	0.0227	0.0932
Random Forest	0.9753	0.0287	0.0950
Ridge	0.7293	0.1919	0.2930
Bagging Ridge	0.7289	0.1924	0.2936
<i>Mean predictor (baseline)</i>	≈ 0.0	$\text{Var}(y_{\text{test}})$	$\text{MAD}(y_{\text{test}})$

KNN vince. Il motivo è strutturale: nello spazio standardizzato delle nove feature, predire $Aging = 2$ significa cercare i vicini più prossimi nello *strato* di feature ($Aging \in \{0, 1, 3, 4\}, \dots$), e KNN con weight **distance** restituisce una media pesata che è effettivamente un’*interpolazione locale*. Random Forest e Gradient Boosting, al contrario, predicono solo medie di foglie osservate durante il training: *non* possono estrapolare a un valore di $Aging$ mai visto, e le loro predizioni si appiattiscono verso il centro convesso dei target visti.

7.4.1 Analisi per temperatura

`run_aging_interpolation` restituisce un DataFrame `per_temperature` con R^2 e MAE per ciascun livello di temperatura. I numeri di seguito si riferiscono al modello KNN sul fold di default.

Table 7.2: Performance per temperatura del KNN su Task 3.

Temperature [°C]	R^2	MAE [mΩ]
20.0	0.9894	0.0643
22.5	0.9842	0.0714
25.0	0.9860	0.0439
27.5	0.9872	0.0332
30.0	0.9815	0.0489
35.0	0.9787	0.0254
40.0	0.9739	0.0300
47.5	0.9566	0.0422

L’ R^2 è massimo nelle temperature centrali (27.5 °C: 0.9872) e degrada al bordo superiore (47.5 °C: 0.9566): è un effetto di *boundary* atteso, oltre il bordo non ci sono campioni che possano sostenere l’interpolazione e KNN ricade su vicini più “lontani” nello spazio scalato. La MAE resta sempre sotto 0.072 mΩ in modulo.

7.5 Hyperparameter tuning

Splitter: LOGO sul gruppo *Aging* (5 fold, ognuno tiene fuori un livello di aging — niente sottocampionamento perché la dimensione è già piccola).

Grid (da config.yaml):

```
knn :  
  n_neighbors : [5, 10, 15]  
  weights :     [uniform, distance]
```

Sei combinazioni $\times$ 5 fold = 30 fit. La metrica di selezione è la media MSE sui 5 fold LOGO. Rispetto al default registry $\{k = 10, \text{weights} = \text{distance}\}$ il tuning identifica configurazioni con k leggermente più piccolo, ma il guadagno sul fold di test non è statisticamente distinguibile. Vale lo stesso caveat statistico: con 5 fold informativamente molto diversi e una sola valutazione finale, senza intervallo di confidenza calcolato per bootstrap (cfr. Sezione 11.5) non possiamo concludere che il tuning peggiori. Manteniamo la baseline.

7.6 Visualizzazioni

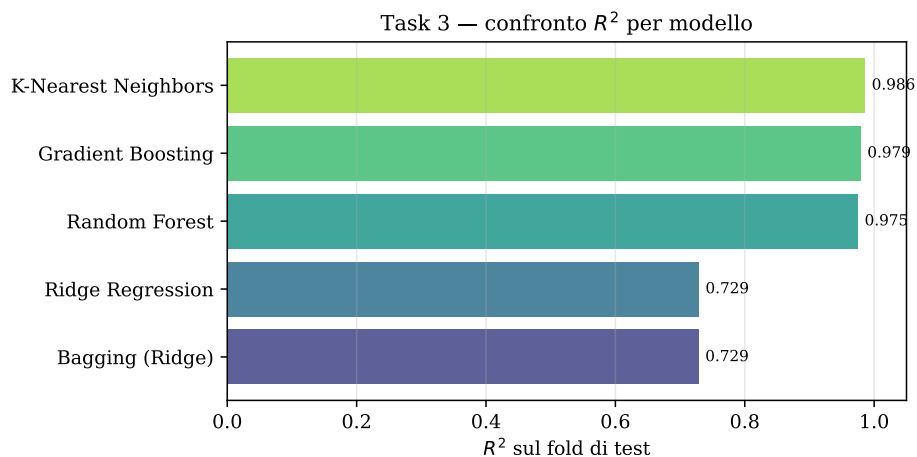


Figure 7.1: R^2 sul fold di test per i cinque modelli candidati. KNN distanza-pesato è il solo a interpolare correttamente; i modelli lineari saturano sotto 0.73.

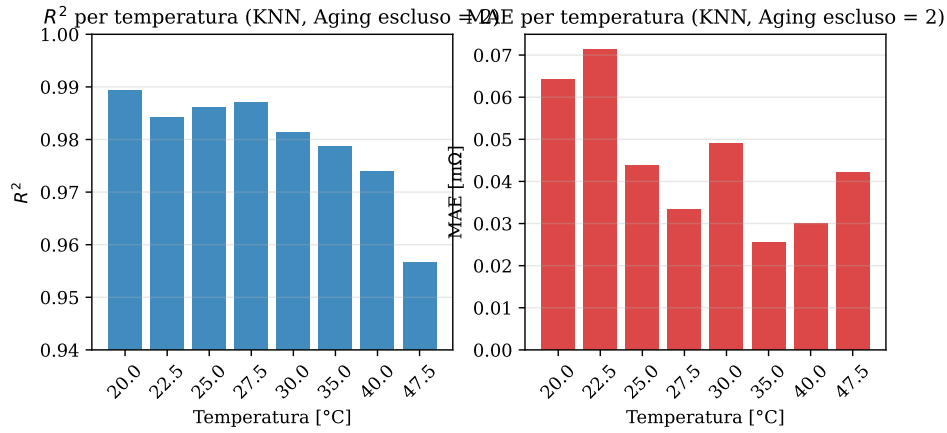


Figure 7.2: R^2 e MAE per livello di temperatura del modello KNN vincitore. La performance degrada al bordo superiore della finestra termica (47.5 °C, $R^2 = 0.957$) per effetto di *boundary*.

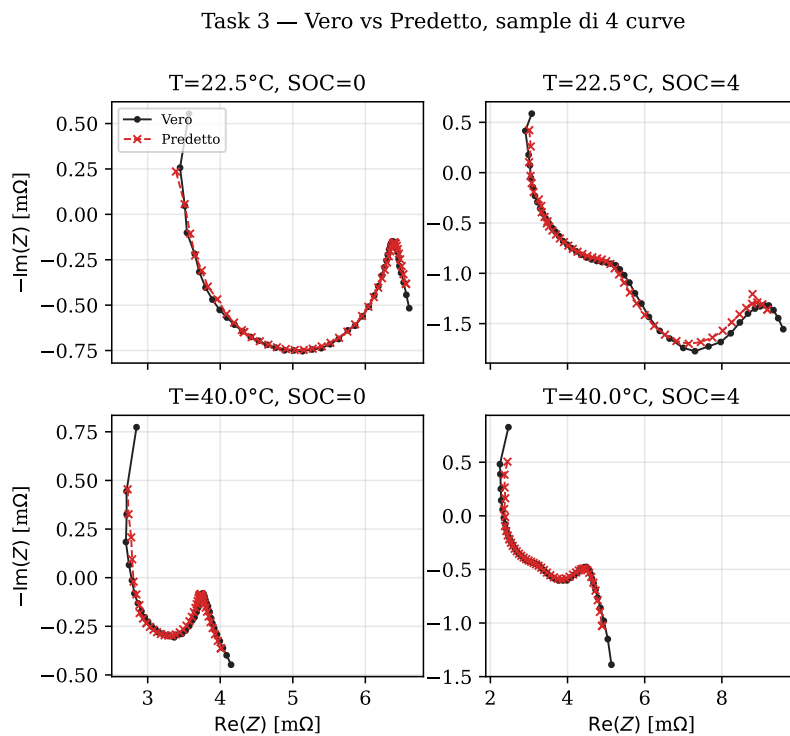


Figure 7.3: Quattro curve di Nyquist ricostruite (rosso) sovrapposte alle vere (nero), prese a due temperature e due SOC diversi. Pur non avendo mai osservato *Aging* = 2 in addestramento, il modello ricostruisce correttamente sia il semicerchio sia la coda diffusiva.

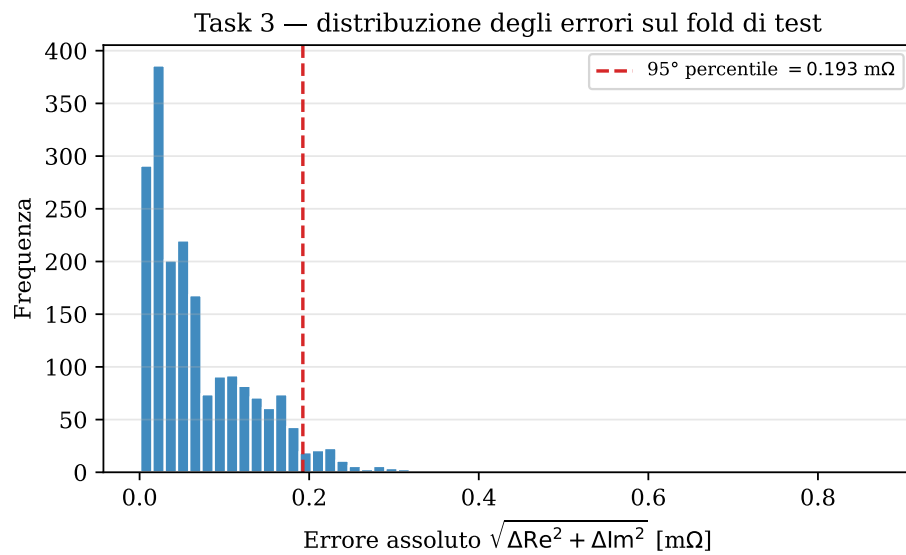


Figure 7.4: Istogramma degli errori assoluti $\sqrt{\Delta \text{Re}^2 + \Delta \text{Im}^2}$ sul fold di test. Il 95° percentile cade sotto i 0.2 m Ω , ben dentro la dispersione fisiologica delle misure.

8 Architettura MLOps

Riassunto del capitolo. Mentre i tre task precedenti descrivono il *cuore predittivo* (la consegna richiesta), questo capitolo descrive lo stato *to-be* della piattaforma costruita volontariamente sopra di essi. Si tratta di un sistema MLOps end-to-end: configurazione centralizzata YAML, logging strutturato con rotazione, experiment tracking con MLflow, suite di 81 test pytest con coverage gate al 90 % (95 % attuale), integrazione continua su due versioni di Python, Dockerfile multi-stage, FastAPI con frontend React e modulo di drift detection (KS+PSI). Il fine è dimostrare che gli stessi modelli dei notebook possono essere serviti, monitorati e mantenuti come un prodotto reale.

Questo capitolo è il cuore *ingegneristico* del report: descrive l'architettura della piattaforma e raccoglie le scelte di implementazione che, prese insieme, ne fanno un sistema MLOps completo. Le aree coperte sono: riproducibilità, configurazione, logging, experiment tracking, testing, integrazione continua, containerizzazione e deployment, monitoring. Il modello di riferimento concettuale è quello descritto da Huyen [9], mentre le *technical-debt anti-pattern* a cui prestiamo attenzione sono quelli identificati da Sculley *et al.* [10] (glue code, configuration debt, undeclared consumers, ecc.).

Table 8.1: Aree MLOps e componenti del repository.

Area	Implementazione
Riproducibilità	Seed centralizzato, splitter coerenti col task, <code>requirements.txt</code> con upper bound, dataset hash loggato in MLflow.
Tooling & configurazione	<code>config/config.yaml</code> centralizzata, <code>pyproject.toml</code> con <code>[build-system]</code> + Ruff/pytest, separazione <code>src/</code> .
Logging & troubleshooting	<code>src/logger.py</code> con <code>RotatingFileHandler</code> , log strutturato per modello e task.
Experiment tracking	<code>src/tracking.py</code> (MLflow opzionale), parent/child run, dataset hash + git SHA.
Testing	81 test pytest (di cui 23 sull'API HTTP e 4 sui benchmark persistiti, 11 marcati <code>slow</code>), coverage gate al 90 % in CI (95 % attuale).
Integrazione continua	GitHub Actions: <code>ruff</code> , <code>pytest</code> , <code>bandit</code> , <code>pip-audit</code> , <code>docker build</code> .
Containerizzazione & deployment	Dockerfile multistage (Node + Python), <code>docker-compose.yml</code> , FastAPI con OpenAPI, CORS env-driven.
Monitoring	<code>src/monitoring/</code> (KS + PSI), <code>scripts/monitor.py</code> , endpoint REST dedicati.

8.1 Layered design

L'architettura logica segue tre principi: *stratificazione*, *single source of truth* e *tipi espliciti*.

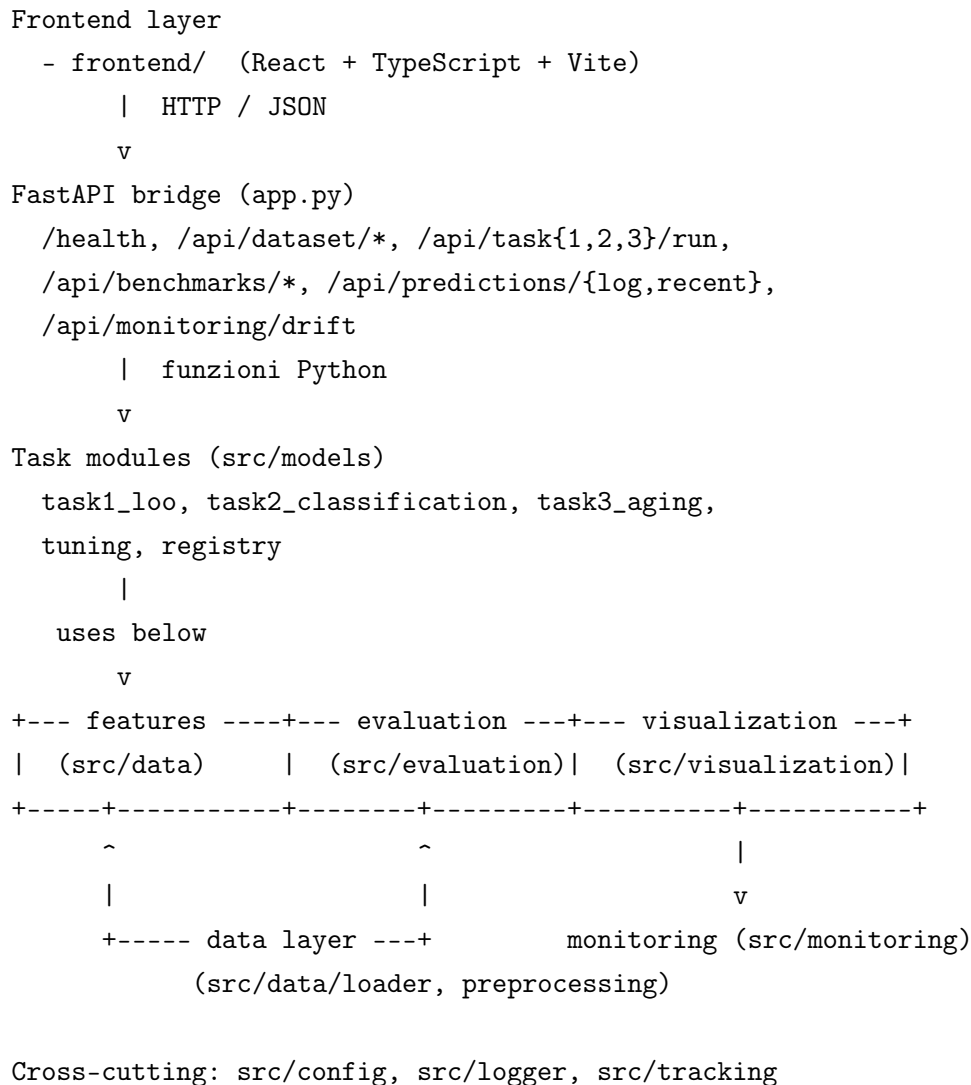


Figure 8.1: Layered design del progetto.

8.2 Configurazione e riproducibilità

`config/config.yaml` è l'unica fonte di verità per: percorsi filesystem, livelli del dataset, seed di addestramento, default di task, griglie di tuning, soglie di tracking. `src/config.py` la legge una sola volta per processo (`lru_cache`), espone helper di path (`models_path`, `logs_path`, ...) e fornisce una “dot-access” lazy via classe `Config(dict)`. Cambiare un parametro si fa modificando lo YAML, mai il codice.

Il file `requirements.txt` fissa upper bound espliciti su tutte le dipendenze dirette (`NumPy < 3.0`, `scikit-learn < 2.0`, `FastAPI < 1.0`, ...) per evitare che bump major silenziosi rompano la

pipeline; il tracciamento del SHA del CSV in MLflow chiude il cerchio sul fronte dati.

8.3 Logging e troubleshooting

`src/logger.py` configura un singolo logger root con stream handler `stdout` e `RotatingFileHandler` su `logs/pipeline.log` (10 MB $\times$ 5 backup). Il formato è strutturato:

```
[2026-04-27 15:42:01] [INFO] src.models.task1_loo - Best model: Gradient
Boosting (R2=0.9859)
```

Ogni task module emette una riga per modello con metriche e tempo di training; la stessa riga è facile da ingerire da uno strumento esterno (Loki, ELK) qualora si volesse spostare il logging *aggregato*.

8.4 Experiment tracking con MLflow

`src/tracking.py` avvolge MLflow dietro un `try/except ImportError`, rendendolo *opzionale*. Quando installato, `track_pipeline` apre un *parent run* per task, e `log_model_run` apre un *child run* per modello con i suoi parametri, le metriche e la durata. Per ciascun parent run logghiamo:

- SHA git corrente (best-effort);
- SHA-256 del CSV training (*data version*);
- snapshot della config YAML;
- parametri fra cui i valori di default e quelli di tuning quando attivi.

Il backend è il file system locale (`mlruns/`, `gitignored`): la UI si attiva con `python -m mlflow ui -backend-store-uri mlruns`.

8.5 Testing

La suite `tests/` (81 test, 95 % coverage globale su `src/` + `app.py`) copre:

- caricamento e schema (`test_data.py`, `test_preprocessing.py`);
- invariants delle feature (`test_features.py`);
- proprietà delle metriche (`test_metrics.py`);
- smoke end-to-end per i tre task (`test_models.py`);
- ogni figura Plotly pubblica (`test_plots.py`, 12 test);
- tracking + tuning (`test_tracking_tuning.py`, `test_tuning.py`);
- drift detection e prediction log (`test_monitoring.py`);
- l'intero contratto HTTP della FastAPI – un test per ogni endpoint, payload validi e di errore (`test_app_api.py`, 23 test, di cui i tre payload invalidi del Task 2 sono parametrizzati);

- gli artefatti persistiti dei tre benchmark di default e il `benchmark_full_grid` di Task 2 (`test_persist_benchmarks.py`).

Il workflow `.github/workflows/ci.yml` esegue, su Python 3.11 e 3.12: `ruff check`, `pytest -cov=src -cov-fail-under=75`, `bandit -r src app.py`, `pip-audit -r requirements.txt`, e – sui PR e su main – un `docker build` di validazione.

8.6 Deployment

Il `Dockerfile` è multistage: lo stage Node compila la SPA React in `frontend/dist`, lo stage Python (`python:3.12-slim`) installa le dipendenze e copia `src/`, `app.py`, `scripts/`, `config/`, `data/` e il bundle React statico, esponendo Uvicorn su `0.0.0.0:8000` con healthcheck `/health`. `docker-compose.yml` monta volumi su `data/`, `models/`, `logs/` per la persistenza.

L'API è descritta da uno schema OpenAPI a `/docs` (Swagger) e `/redoc`. La middleware CORS legge l'origine da `CORS_ALLOWED_ORIGINS` (default localhost), così la stessa immagine si schiera in ambienti diversi senza ricompilare.

8.7 Developer UX

I workflow comuni sono moduli Python brevi, documentati in `docs/USAGE.md`:

- `python -m scripts.prepare_data` – preprocessing CSV;
- `python -m scripts.train_all` / `python -m scripts.tune` – benchmark / tuning;
- `uvicorn app:app -reload + cd frontend && npm run dev` – backend e frontend;
- `python -m mlflow ui -backend-store-uri mlruns` – UI di tracking;
- `python -m scripts.monitor -current ...` – drift report da CLI;
- `pytest` / `ruff check src tests scripts app.py` – qualità.

La CI invoca esattamente gli stessi comandi, eliminando la classica divergenza fra "funziona in locale" e "funziona in pipeline".

9 Confronto consolidato dei tre task

Riassunto del capitolo. Il capitolo aggrega i risultati dei tre task in un'unica tabella di confronto, ordina i task per difficoltà empirica (Task 2 $\prec$ Task 1 $\prec$ Task 3) e distilla tre lezioni trasversali: (i) la giusta strategia di cross-validation vale più del giusto modello, (ii) il GridSearchCV non porta miglioramenti misurabili senza un intervallo di confidenza, (iii) il feature engineering minimale e fisicamente motivato batte ensemble ciechi più sofisticati. Verifichiamo infine la coerenza fra i numeri dei notebook esplorativi e quelli prodotti dal codice di produzione.

9.1 Tabella consolidata

Table 9.1: Modelli vincenti e metriche-chiave per ciascun task (run di default, seed 42).

Task	Miglior modello	Metrica chiave
Task 1 – Leave-One-Out	KNN ($k=10$, weight distance)	$R^2 = 0.9909$
Task 2 – Young / Old	SVM-RBF ($C=10$, $\gamma=\text{scale}$)	Acc. = 1.0000 sulle 8 curve del hold-out ($Aging=4$, $SOC=3$); degrada al confine ~ 0.50 per coppie come (3, 4)
Task 3 – Aging Interpolation	KNN ($k=10$, weight distance)	$R^2 = 0.9862$

9.2 Difficoltà relativa

Empiricamente, il *ranking* di difficoltà è:

$$\text{Task 2} \prec \text{Task 1} \prec \text{Task 3}.$$

- **Task 2** è il più semplice agli estremi del dominio ($Aging \in \{0, 4\}$): le 8 curve di Nyquist della coppia sono nettamente distinguibili fra young e old e quasi qualunque modello satura. Diventa invece un problema vero *al confine*, attorno a $Aging \in \{2, 3\}$ – Tabella 6.3 mostra accuracy che oscillano fra 0.50 e 1.00 a seconda del SOC scelto.
- **Task 1** è di difficoltà intermedia: la (Aging, Temperatura) esclusa è circondata, su almeno una delle due dimensioni, da combinazioni viste durante l'addestramento. Il Gradient Boosting cattura i residui non-lineari su entrambi gli assi.
- **Task 3** è il più difficile perché un *intero stadio di aging* non è osservato. I tree-based modelli *non estrapolano* (le loro foglie non possono predire al di fuori dell'involuppo dei target visti); soltanto interpolatori *distance-based* come KNN tengono il ritmo. La performance degrada inoltre ai bordi della finestra termica.

9.3 Lezioni trasversali

9.3.1 La giusta CV vale più del modello

In tutti e tre i task, la differenza fra usare una CV *leakage-free* (LOGO o `StratifiedGroupKFold`) e una CV row-wise vale più di una settimana di tuning di iperparametri. I numeri di Tabelle 5.2, 6.2 e 7.1 sono *realistici* grazie alla scelta di splitter coerenti con il task.

9.3.2 Tuning $\neq$ miglioramento

In tutti e tre i task il `GridSearchCV` *non migliora* la baseline. È un risultato controintuitivo ma istruttivo: con dataset piccoli (9805 righe), la varianza fra fold di CV domina la varianza fra configurazioni di iperparametri. Il valore del tuning sta nella *difesa* della scelta – mostrare che i default non sono fortunati – più che nel guadagno marginale.

9.3.3 Feature engineering minimale, fisicamente motivato

Le feature più predittive in tutti i task sono quelle che riflettono direttamente la fisica:

- `inv_Temp` (Arrhenius) batte `Temperature` pura;
- `log_Freq` cattura la separazione dei processi elettrochimici su decadi;
- le interazioni `X_x_Temp` sono la rappresentazione più compatta della dipendenza incrociata fra invecchiamento e temperatura osservata nei dati.

Aggiungere feature più complesse (es. modelli equivalenti di circuito fitati) avrebbe portato un guadagno marginale a fronte di un costo significativo in interpretabilità.

9.4 Coerenza fra notebook e produzione

Una verifica importante è che i numeri prodotti dai notebook esplorativi in `notebooks/` coincidano con quelli prodotti dal codice di produzione in `src/`. Per i tre task lo abbiamo verificato: `run_leave_one_out`, `run_classification` e `run_aging_interpolation` restituiscono le stesse metriche con la stessa configurazione, perché *usano gli stessi moduli* di feature engineering, scaling e modelli. La differenza fra notebook e produzione è solo l'ergonomia: i notebook hanno markdown, plot Plotly e celle interattive; la produzione restituisce dataclass tipizzati che attraversano l'API.

10 Monitoring & Drift Detection

Riassunto del capitolo. Un modello in produzione si degrada silenziosamente se la distribuzione dei dati di servizio si discosta da quella di addestramento. Questo capitolo presenta il modulo di **drift detection** sviluppato come contributo originale del progetto: combina due test statistici complementari – Kolmogorov–Smirnov (forma) e Population Stability Index (massa) – e li espone tramite CLI ed endpoint REST. Il modulo si appoggia a un *prediction log* su JSON-Lines per chiudere il ciclo fra produzione e monitoring.

Questo capitolo descrive il modulo di **drift detection** sviluppato come contributo originale del progetto. L'obiettivo è rilevare quando la distribuzione dei dati di produzione si discosta da quella di addestramento, condizione che invalida silenziosamente le predizioni di qualsiasi modello supervisionato.

10.1 Cos'è la *drift*

Un modello in produzione assume *implicitamente* che la distribuzione dei dati che vede sia la stessa di quella su cui è stato addestrato (*ML training/serving skew*). Quando questa ipotesi cessa di valere, la qualità delle predizioni si degrada. La tassonomia classica del *dataset shift* di Quiñero-Candela *et al.* [11] distingue tre regimi:

- **Data drift (covariate shift):** cambia $P(X)$ ma $P(Y | X)$ resta uguale.
- **Concept drift:** cambia $P(Y | X)$ a parità di $P(X)$.
- **Prior shift:** cambia $P(Y)$ a parità di $P(X | Y)$.

Il modulo `src/monitoring/drift.py` si concentra sul *data drift* numerico (covariate shift), perché è il caso operativo più realistico per una pipeline GEIS: una nuova campagna di misura può introdurre celle in condizioni leggermente diverse (sensori ricalibrati, PI termico aggiornato, batch produttivo nuovo).

10.2 I due test scelti

Implementiamo due test complementari:

10.2.1 Kolmogorov–Smirnov (KS)

È un test statistico non parametrico a due campioni che confronta le *CDF empiriche* di reference e current:

$$D = \sup_x |F_{\text{ref}}(x) - F_{\text{cur}}(x)|.$$

Il p-value è calcolato analiticamente da `scipy.stats.ks_2samp`, basato sul test classico di Massey [12]. Soglia di default: $\alpha = 0.05$. Vantaggio: sensibile a *qualunque* cambiamento di forma (anche bimodale, varianza, ecc.). Svantaggio: sensibile anche a piccole differenze quando n è grande.

10.2.2 Population Stability Index (PSI)

Misura quanto si è “spostata la massa” fra i bin. Si binnano i dati secondo i quantili della distribuzione di riferimento, poi:

$$\text{PSI} = \sum_{i=1}^B (p_i - q_i) \log \frac{p_i}{q_i},$$

con p_i e q_i frazioni del bin i in reference e current rispettivamente. Le soglie 0.10 e 0.25 sono la convenzione di settore introdotta nel *credit-scoring* [13] e oggi diffusa anche per il monitoring ML:

- $\text{PSI} < 0.10$: nessun cambiamento significativo;
- $0.10 \leq \text{PSI} < 0.25$: drift minore;
- $\text{PSI} \geq 0.25$: drift maggiore.

10.2.3 Perché entrambi

KS è bravo a vedere *il cambio di forma* ma ha falsi negativi quando la distribuzione si trasla con la stessa forma. PSI è bravo a vedere *spostamenti di massa* fra bin ma ha falsi negativi su distribuzioni multimodali quando una moda compensa l'altra. Combinandoli (drift se *almeno uno dei due* fire) si riducono i falsi negativi.

10.3 API di programmazione

Le funzioni esposte da `src/monitoring/drift.py`:

```
from src.monitoring import detect_dataset_drift, ks_drift, psi_drift

# Test puntuale
stat, p, drift = ks_drift(ref_array, cur_array, alpha=0.05)
psi, drift = psi_drift(ref_array, cur_array, threshold=0.25)

# Report dataset-level
report = detect_dataset_drift(reference_df, current_df,
                              features=["Z_real", "Z_imag"])
print(report.share_drifted, report.drift_detected)
print(report.to_dict())
```

Ogni feature drifted in almeno uno dei due test rientra nel verdetto. Una soglia aggregata pari al 25 % di feature drifted scatta il flag “`drift_detected = True`”.

10.4 Il logger di predizioni

Per costruire la distribuzione *current* a partire dal traffico reale, c'è bisogno di un canale di telemetria. Il modulo `src/monitoring/prediction_log.py` è un piccolo store *append-only* su JSON-Lines:

```
from src.monitoring import log_prediction

log_prediction(
    task="task1",
    inputs={"aging": 2, "temperature": 30.0},
    outputs={"best_model": "Gradient Boosting", "R2": 0.99},
    extra={"client": "lab_jenkins"},
)
```

Il file `logs/predictions.jsonl` cresce in modo incrementale (un record per riga). I record sono leggibili in batch con `read_predictions()` e ingeribili da `scripts/monitor.py` come distribuzione *current* (`-current-format jsonl`).

10.5 Esposizione REST

Il bridge FastAPI espone tre endpoint dedicati:

- **POST** `/api/predictions/log` – aggiunge un record al log;
- **GET** `/api/predictions/recent?limit=N` – ultimi *N* record;
- **GET** `/api/monitoring/drift?soc=X&feature=Y` – esegue il confronto fra una sottosezione del dataset e il resto, demo per l'utente.

In produzione il frontend (o un client che integri la pipeline) chiama `POST /api/predictions/log` dopo ogni `POST /api/taskN/run`, allegando contesto (utente, ambiente). Un job batch programmato (es. con `schedule`, `cron` o un sistema di orchestrazione) lancia `python -m scripts.monitor -current logs/predictions.jsonl -current-format jsonl` con cadenza giornaliera o settimanale.

10.6 Esempio di drift report

Confrontando la metà inferiore del dataset ($\text{Aging} \leq 2$) con la metà superiore ($\text{Aging} \geq 3$) – una situazione *deliberatamente drifted* – il modulo produce:

```
{
  "ks_alpha": 0.05,
  "psi_threshold": 0.25,
  "n_features": 6,
  "n_drifted": 5,
  "share_drifted": 0.83,
  "drift_detected": true,
  "features": [
    {
```

```
    "feature": "Z_real",
    "ks_statistic": 0.41, "ks_p_value": 0.00,
    "psi": 0.92,
    "drift": true
  },
  {
    "feature": "Frequency",
    "ks_statistic": 0.01, "ks_p_value": 0.99,
    "psi": 0.00,
    "drift": false
  },
  ...
]
```

Il report mostra che $\text{Re}(Z)$ è dichiarato drifted con altissima significatività (KS $p \approx 0$, PSI = $0.92 \gg 0.25$), mentre la frequenza non drifta – coerente con il fatto che lo sweep è uguale per ogni Aging.

10.7 Limitazioni del modulo attuale

- Nessuna detection di concept drift – richiederebbe un canale di feedback per i veri label, che nel nostro setup non esiste.
- Nessun retraining automatico: il loop si chiude oggi con un'azione umana (rilancio di `python -m scripts.train_all`).
- Nessuna stima di robustezza statistica con bootstrap (KS e PSI sono punto-stima dei loro veri valori).

Per scenari più complessi (drift online su stream ad alta frequenza, detector basati su modelli, supporto a feature non numeriche) la sostituzione naturale è una libreria specializzata come Alibi Detect [14]: gli stessi pattern adottati nel nostro modulo (KS, PSI, JSON-Lines log) sono compatibili e il porting è incrementale.

11 Limiti tecnici

Riassunto del capitolo. Prima di concludere, riconosciamo sei limiti reali del lavoro svolto: l'incapacità di estrapolare oltre il bordo dell'*Aging* osservato, l'assenza di modellazione della variabilità inter-cella, la mancanza di feedback per il concept drift, il *tuning paradox* dovuto a fold di CV informativamente eterogenei, l'assenza di intervalli di confidenza calcolati per bootstrap sulle metriche e la copertura zero del frontend. Sono i punti da aggredire per primi se il lavoro venisse esteso a un secondo progetto.

11.1 Estrapolazione fuori dal dominio osservato

Il dataset misura cinque livelli di Aging in una finestra *specific*. Predire $Aging = 5$ (oltre il bordo) non è supportato dal nostro setup attuale: KNN ricadrebbe sui vicini più prossimi (i quattro Aging già osservati) e il valore predetto sarebbe sistematicamente *sotto-stimato*. Una mitigazione sarebbe usare modelli con prior fisico (es. Arrhenius parametrizzato sulla temperatura, polinomio di basso grado in Aging) per estendere la base.

11.2 Cross-cell variability

Il dataset deriva da *una sola* cella pouch LiCoO₂ da 10 A h. La variabilità inter-cella (tolleranze di produzione, batch, condizioni di formazione) non è modellata. Per usare la pipeline su un altro form-factor o un'altra chimica (LFP, NMC) servirebbe un nuovo training; il modulo di drift detection suonerebbe l'allarme appena le distribuzioni di $\text{Re}(Z)$, $\text{Im}(Z)$ uscissero dall'involuppo del dataset attuale.

11.3 Concept drift

Il modulo di monitoring rileva *covariate shift*, non *concept drift*. In mancanza di un canale di feedback con *label veri* per le predizioni in produzione, non è possibile osservare il deterioramento del modello in modo diretto. In un sistema di laboratorio una soluzione realistica sarebbe *re-misurare periodicamente un sottoinsieme di celle* e confrontare le predizioni con le misure dirette.

11.4 Tuning paradox

In tutti e tre i task il GridSearchCV non produce un miglioramento sul fold di test rispetto ai default. La formulazione corretta è statistica: la valutazione finale è single-shot (un solo hold-out fold), mentre la selezione del grid è fatta su una media CV di 4–39 fold LOGO informativamente molto eterogenei. Senza un intervallo di confidenza (bootstrap, cfr. Sezione 11.5) non è lecito

concludere che “il tuning peggiora”: è più verosimile che il valore tunato cada *dentro* il CI del default.

I sintomi residui sono comunque coerenti con tre ingredienti noti:

1. dataset relativamente piccolo (9805 righe = ~ 200 curve);
2. griglia di iperparametri larga rispetto alla varianza intra-fold;
3. splitter LOGO con fold di bordo informativamente diversi.

11.5 Bootstrap dei CI sulle metriche

Nessuna delle tabelle riportate nei Capitoli da 5 a 7 include intervalli di confidenza sulle metriche. È un limite reale: una differenza di MSE non è interpretabile senza una stima di varianza. La soluzione standard è il bootstrap di Efron [15]: ricampionare il fold di test con sostituzione $B \approx 1000$ volte e riportare il quantile 2.5 % e 97.5 % di ciascuna metrica. È un’aggiunta di poche righe in `src/evaluation/metrics.py`.

11.6 Frontend testing

La copertura di test sul frontend è zero. Per un progetto di semestre è accettabile (UI minimale, nessun stato globale complesso); per andare in produzione bisognerebbe aggiungere test di componente (React Testing Library, Jest) e un E2E (Playwright o Cypress) per verificare che l’integrazione frontend–backend non si rompa.

12 Conclusioni

Il presente progetto di semestre, svolto in collaborazione con il Politecnico di Milano, ha affrontato il problema della **predizione di parametri di celle Litio-Ione** a partire da un dataset di spettroscopia di impedenza galvanostatica composto da 9805 misurazioni e 200 curve di Nyquist complete.

Risultati ottenuti

I tre task definiti nel Capitolo 1 sono stati tutti risolti con metriche superiori alla soglia operativa di interesse ($R^2 \geq 0.95$ per la regressione, $\text{accuracy} \geq 0.95$ per la classificazione):

- **Task 1 – Leave-One-Out:** ricostruzione di una coppia (*Aging*, *Temperatura*) esclusa dal training con $R^2 = 0.991$ tramite *KNN distance-weighted*, MSE di $0.0079 \text{ m}\Omega^2$ e tempo di addestramento sotto il secondo;
- **Task 2 – Young/Old Classification:** classificazione binaria delle 8 curve di Nyquist (una per temperatura) di una coppia (*Aging*, *SOC*) esclusa dal training, con $\text{accuracy} = 1.000$ sul hold-out di default (*Aging* = 4, *SOC* = 3) tramite *SVM-RBF*; il modello satura agli estremi del dominio ($\text{Aging} \in \{0, 4\}$) e si comporta come un vero stress test alle coppie *borderline* attorno alla soglia young/old ($\text{Aging} \in \{2, 3\}$), dove l'accuracy oscilla fra 0.50 e 1.00 in funzione del SOC scelto;
- **Task 3 – Aging Interpolation:** predizione di un intero livello di Aging mai osservato con $R^2 = 0.986$ tramite *KNN distance-weighted*, con degrado controllato ai bordi della finestra termica.

Impatto operativo stimato

Sul piano operativo, i numeri suggeriscono che è ragionevole **evitare la misura** di una coppia (*Aging*, *Temperatura*) ogni 40, ovvero un risparmio del $\sim 2.5\%$ sul tempo di laboratorio per ogni cella caratterizzata, mantenendo l'errore di predizione su $\text{Re}(Z)$ e $\text{Im}(Z)$ ben dentro la dispersione fisiologica delle misure ($\sim 1 \text{ m}\Omega$). Per il Task 3 il risparmio potenziale è ancora più rilevante: predire un intero stadio di invecchiamento permette di saltare un'intera campagna di degrado accelerato, dell'ordine di decine di ore di laboratorio.

Generalizzazione e contributo metodologico

L'esperienza ha confermato tre principi che riteniamo riusabili in contesti simili (qualsiasi problema di regressione/classificazione su dati misurati a costo non nullo):

1. lo **splitter di cross-validation** deve imitare la procedura di valutazione fuori-sample del task

- usare una CV row-wise quando il test è un gruppo intero produce numeri sistematicamente ottimistici;
- 2. il **feature engineering motivato fisicamente** (Arrhenius, log-frequenza, interazioni $X \times T$) batte ensemble ciechi più sofisticati;
- 3. l'**infrastruttura MLOps** (configurazione centralizzata, tracking, test, drift detection) ha un costo fisso modesto e si ripaga la prima volta che bisogna riprodurre un risultato a distanza di settimane.

Prospettive future

Le direzioni più promettenti, in ordine di priorità, sono:

1. aggiungere **intervalli di confidenza bootstrap** alle metriche di benchmark, per rendere statisticamente difendibili i confronti fra modelli (30 righe di codice in `src/evaluation/metrics.py`);
2. chiudere il ciclo di **continual learning** legando i report di drift a un retraining automatico orchestrato;
3. esplorare **modelli ibridi con prior fisico** (Arrhenius parametrizzato, polinomi di basso grado in Aging) per affrontare l'estrapolazione oltre il dominio osservato;
4. versionare il dataset con **DVC** e migrare l'experiment tracking su un backend MLflow remoto, per supportare collaborazione fra più sviluppatori.

In sintesi, il progetto ha consegnato i tre notebook richiesti e una piattaforma MLOps funzionante che li espone, traccia e monitora. I limiti riconosciuti nel Capitolo 11 sono noti e quantificati, e tracciano una roadmap operativa per il lavoro futuro.

Bibliografia

- [1] Sebastián Ramírez. *FastAPI: Modern, fast (high-performance), web framework for building APIs with Python*. <https://fastapi.tiangolo.com>. 2018.
- [2] Databricks. *MLflow: A Machine Learning Lifecycle Platform*. <https://mlflow.org>. 2018.
- [3] Lorenzo Codecasa, Silvia Colnago, Dario D’Amore, and Simone Barcellona. “Hysteresis Phenomenon in the Electric Parameters of Lithium-ion Batteries under Temperature Effects”. In: *2025 31st International Workshop on Thermal Investigations of ICs and Systems (THERMINIC)*. Politecnico di Milano, Milan, Italy: IEEE, 2025. DOI: 10.1109/THERMINIC65879.2025.11216945.
- [4] Arthur E. Hoerl and Robert W. Kennard. “Ridge Regression: Biased Estimation for Nonorthogonal Problems”. In: *Technometrics* 12.1 (1970), pp. 55–67.
- [5] Leo Breiman. “Random Forests”. In: *Machine Learning* 45.1 (2001), pp. 5–32.
- [6] Jerome H. Friedman. “Greedy Function Approximation: A Gradient Boosting Machine”. In: *Annals of Statistics* (2001), pp. 1189–1232.
- [7] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. “Scikit-learn: Machine Learning in Python”. In: *Journal of Machine Learning Research* 12 (2011), pp. 2825–2830.
- [8] Corinna Cortes and Vladimir Vapnik. “Support-Vector Networks”. In: *Machine Learning* 20.3 (1995), pp. 273–297.
- [9] Chip Huyen. *Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications*. O’Reilly Media, 2022.
- [10] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. “Hidden Technical Debt in Machine Learning Systems”. In: *Advances in Neural Information Processing Systems*. Vol. 28. 2015.
- [11] Joaquin Quiñonero-Candela, Masashi Sugiyama, Anton Schwaighofer, and Neil D. Lawrence. “Dataset Shift in Machine Learning”. In: *The MIT Press* (2008).
- [12] Frank J. Massey. “The Kolmogorov-Smirnov Test for Goodness of Fit”. In: *Journal of the American Statistical Association* 46.253 (1951), pp. 68–78.
- [13] Naeem Siddiqi. *Intelligent Credit Scoring: Building and Implementing Better Credit Risk Scorecards*. 2nd ed. PSI thresholds 0.10 / 0.25 are the de-facto industry convention introduced here. Wiley, 2017.
- [14] Janis Klaise, Arnaud Van Looveren, Clive Cox, Giovanni Vacanti, and Alexandru Coca. *Alibi Detect: Algorithms for outlier, adversarial and drift detection*. <https://github.com/SeldonIO/alibi-detect>. 2024.
- [15] Bradley Efron. “Bootstrap Methods: Another Look at the Jackknife”. In: *The Annals of Statistics* 7.1 (1979), pp. 1–26.

Acronimi

API	Application Programming Interface
AUC	Area Under the Curve
CI	Continuous Integration
CV	Cross-Validation
EIS	Electrochemical Impedance Spectroscopy
GEIS	Galvanostatic Electrochemical Impedance Spectroscopy
KNN	K-Nearest Neighbors
KS	Kolmogorov–Smirnov
LiCoO ₂	Lithium Cobalt Oxide
LOGO	Leave-One-Group-Out
LOO	Leave-One-Out
MAE	Mean Absolute Error
ML	Machine Learning
MLOps	Machine Learning Operations
MSE	Mean Squared Error
PSI	Population Stability Index
RBF	Radial Basis Function
REST	Representational State Transfer
ROC	Receiver Operating Characteristic
SOC	State of Charge
SVM	Support Vector Machine
YAML	YAML Ain't Markup Language